Contents

Expanding incremental-model eligibility

Status: research (decision-oriented, living document) **Date started:** 2026-07-01 **Owners:** andrew **Related:** - Spec: `docs/specs/incremental_models.md` - Plan: `docs/plans/20260701-monotonicity-primitive.md` (implements Part 6) - `docs/research/2026-05-20-incremental-gaps-from-web-analytics.md` - `docs/research/20260521-incremental-as-planner-rule.md`

Why this document exists

smelt currently **rejects** a model from incremental materialization in a large number of situations. Some of those rejections are genuine correctness requirements; others are *conservative* — the general case is unsafe, but a well-characterised sub-case is provably fine and is exactly the pattern users reach for. Every unnecessary rejection pushes a user onto a full-table rebuild (or off smelt), so the rejections are worth auditing one by one.

This document is the shared, growing home for that audit. It opens with a **catalogue** of everything that currently disqualifies a model, then works through the conditions one at a time — for each, asking *why* it is rejected, whether the rejection is a correctness law or a mechanical limitation, and what a safe relaxation would require. The worked conditions so far are **set operations at the base (`UNION ALL`)** (Part 2), **subqueries in `FROM`** (Part 3), and **joins** (Part 5) — the last being the mirror-image case: a construct that is *never* gated yet is not universally safe. Part 4 pulls out the cross-cutting placement question (where to inject a proven-safe filter); **Part 6** is the full deep-dive on the one analysis all three conditions block on — the **monotonicity primitive**; and **Part 7** validates the whole audit against the academic theory of incremental maintenance and the published eligibility rules of production engines (Databricks Enzyme, Snowflake, BigQuery, Flink, Materialize, DBSP). **Part 8** works the **window-function / `LAG`/`LEAD` / two-layer-lookback** cluster (B1/C1) — the one construct where the scan window and the output window legitimately diverge, and the case where smelt can *derive* the lookback margin that streaming engines make users declare — and **Part 9** dispatches the remaining shorter conditions (non-deterministic functions, non-additive aggregates, `HAVING`/`DISTINCT`/`LIMIT`, and `partition_column`-in-`GROUP BY`) by applying the arguments already established rather than discovering new ones. With Part 9 the original rejection catalogue (Part 1) is fully worked; what remains is turning the settled analysis into specs and plans, beginning with the monotonicity primitive (now specified in `docs/specs/incremental_models.md` and planned in `docs/plans/20260701-monotonicity-primitive.md`). Finally, **Part 10** records a constraint the

original catalogue never named — the alignment between the output partition's *granularity* and the run cadence, surfaced by the aggregate-daily-input-to-a-monthly-output use case, and orthogonal to every eligibility gate above — and **Part 11** names a second orthogonal, execution-time axis: whether a model's windows may be run out of order / in parallel (*window-independent*) or must run as a strict forward sequence (*ordered*), the latter forced only by a model reading its own prior output (self-reference / cumulative state).

The governing correctness contract throughout is the **incremental ≡ full-refresh invariant**: running a model as a sequence of adjacent windows must produce the same stored result, partition for partition, as recomputing the whole range in one shot.

Part 1 — Catalogue of current rejections

There are **five enforcement pathways**, with different severity and fallback behaviour. The authoritative implementations live in `smelt-logical` (`smelt-planner` re-exports them).

Pathway A — planner safety check (`incremental::detect`)

`crates/smelt-logical/src/rules/incremental.rs`, driven from `smelt-runtime/src/execute.rs` via `check_planner_safety` (`crates/smelt-runtime/src/safety.rs`). With `enforce_safety = true` (default), a returned `Err` is a **hard error, build refused**. With `--allow-downgrade` (`enforce_safety = false`) it becomes a **`warn!` + silent full-table refresh**.

#	Condition	Site	Overridable via
A1	`incremental:` present, `timeseries:` missing	`incremental.rs:130`	(D1)
A2	SQL not parseable by `analyze_select`	`incremental.rs:146`	—
A3	`partition_column` not a SELECT-list alias	`incremental.rs:158`	—
A4	`partition_column` not in `GROUP BY` (aggregate models)	`incremental.rs:181`	—
A5	`event_time_column` not referenced in SQL	`incremental.rs:195`	—
A6	`unique_key` column not a SELECT-list alias	`incremental.rs:214`	—
B1	window `OVER` whose `PARTITION BY` ⊉ `partition_column`	`incremental.rs:231`	`allow_window_functions`
B2	`HAVING`	`incremental.rs:248`	`allow_having`
B3	`LIMIT`	`incremental.rs:261`	`allow_limit`
B4	subquery in `FROM`	`incremental.rs:273`	`allow_subqueries`
B5	non-deterministic function (`RANDOM`/`NOW`/`UUID`/…)	`incremental.rs:288`	`allow_nondeterministic`
B6	`SELECT DISTINCT`	`incremental.rs:302`	`allow_distinct`

The B-group all carry a per-model `incremental.safety_overrides.allow_*` escape hatch (`crates/smelt-core/src/config.rs:429`); the A-group do not.

Pathway C — temporal-bound derivation (`check_bound_derivation`)

#	Condition	Site	Behaviour
C1	bare LAG/LEAD (window fn without RANGE BETWEEN INTERVAL) → NotDerivable bound	incremental.rs:574 / safety.rs:100; detection analysis/source_bounds.rs:240	hard error / downgrade

Pathway D — frontmatter/metadata validation (workspace load)

#	Condition	Site	Code
D1	incremental: without timeseries:	crates/smelt-core/src/metadata.rs:411	TimeseriesRequiredForIncreme
D2	partition_column not projected / timeseries on ephemeral/test	metadata.rs:362	MalformedTimeseries
D3	incremental config on ephemeral / refresh: cumulative	config.rs:842	—

Pathway E — event-time injectability gate (detect_builtin_rules)

crates/smelt-logical/src/rules/rule_diagnostics.rs, surfaced as **Error diagnostics** through smelt-db → LSP and enforced at the runtime pre-execute gate. **Not** bypassed by --allow-downgrade.

#	Condition	Site	Code
E1	**UNION/INTERSECT/EXCEPT at the outer query**	rule_diagnostics.rs:186	EventTimeColumnNotVisibleAtO
E2	bare subquery FROM not projecting event_time_column	rule_diagnostics.rs:201	EventTimeColumnNotVisibleAtO

Not a rejection (recorded to avoid confusion)

- **Unbounded lookback** (UNBOUNDED PRECEDING, correlated subquery, default window frame) → BatchSafety::PerPartitionOnly (incremental.rs:72). The model **still builds**, just per-partition. Contrast with C1, which *does* reject.
- **IncrementalNotBatchSafe** (rule_diagnostics.rs:154) wraps the Pathway-A errors as a Warning for editor parity; it never blocks on its own.
- **Joins** — neither the inline FROM a JOIN b spelling nor the declared joins: frontmatter has **any** incremental-eligibility gate. A join model builds today with no join-specific check, and inject_source_filters will silently time-window *every* bounded source in the join. This is not a benign non-rejection like the two above: some join shapes are genuinely unsafe, so the missing gate is a latent soundness hole, not a conservative allowance. Worked in **Part 5**.
- **Scalar subqueries over bounded sources** *(recorded 2026-07-02, not yet worked)* — an uncorrelated scalar subquery reading a timeseries ref (SELECT …, (SELECT MAX(ts) FROM smelt.silver.events) AS hwm) has no gate (B4/E2 only look at the FROM clause), and because inject_source_filters windows every smelt.<path> occurrence *textually* (§3.4), the ref inside the subquery is silently time-windowed — turning a global aggregate into a per-window one, the same silent-misfilter shape as the §5.2 join hazard. (Correlated subqueries are separately caught by the unbounded-lookback → PerPartitionOnly path above.)
- **GROUPING SETS / ROLLUP / CUBE** *(recorded 2026-07-02, not yet worked)* — super-aggregate rows carry NULL in the grouping columns, so a GROUP BY ROLLUP(partition_col, …) passes a textual A4 (partition_column appears in the GROUP BY) while emitting rows whose partition column is NULL and whose value aggregates *all* windows — the P3 NULL-event-time hazard

(§2.5) in aggregate form, and the classical empty-grouping-set pushdown caveat (see the PrestoDB reference).

Part 2 — Condition deep-dive: set operations at the base (E1)

2.1 Why it is rejected

Incrementalisation works by injecting `WHERE event_time >= start AND < end`. `inject_time_filter` (`crates/smelt-runtime/src/transformer.rs:272`) finds the **outermost** SELECT's `WHERE` (or `FROM`) and appends the predicate there. On `A UNION ALL B`, a trailing `WHERE` binds to branch `B` only — branch `A` is left unfiltered, silently producing wrong data. Rather than mis-filter, `E1` (`rule_diagnostics.rs:186`, triggered by `SelectStmt::has_set_operation()`) refuses the model up front and tells the user to rewrite it by hand as a CTE/subquery.

So the rejection is a **mechanical limitation of the injection point**, not a statement that set operations are semantically incompatible with incremental refresh.

2.2 The correctness result

Time-filtering is a per-row predicate σ on a **projected** column (`event_time`). Selection distributes over bag-union:

$\sigma(A \uplus B) = \sigma(A) \uplus \sigma(B)$

and, because identical rows share every column value — including `event_time` and `partition_column`, so duplicates always fall in the same window and the same partition — the same distribution holds for `UNION` (distinct), `INTERSECT`, and `EXCEPT`:

$\sigma(A \cup B) = \sigma(A) \cup \sigma(B)$ $\quad$ $\sigma(A \cap B) = \sigma(A) \cap \sigma(B)$ $\quad$ $\sigma(A \setminus B) = \sigma(A) \setminus \sigma(B)$

The `EXCEPT` case deserves a note: a `B` row can only cancel an `A` row when the two are identical *including* `event_time` (set operations compare all projected columns, and `event_time` must be projected). So filtering `B` by the same window never removes a canceller that a full refresh would have kept — there is no cross-window cancellation hazard.

Conclusion: for all four set operations the blocker is purely how the filter is applied, not the algebra.

2.3 Empirical confirmation (DuckDB v1.4.4)

Harness: `docs/research/harness/20260701-union_incremental.sql` (run with `duckdb -box < …`). Each property reports violating rows via `|(L EXCEPT ALL R) ⊎ (R EXCEPT ALL L)|`; `0` = the identity holds.

Property	Violations	Meaning
P1 $\sigma(big \uplus small) = \sigma(big) \uplus \sigma(small)$	**0**	filter distributes over `UNION ALL`
P2 two adjacent windows = full refresh	**0**	incremental ≡ full for `UNION ALL`
P3 branch with `NULL event_time`	**1**	hazard reproduced (see §2.5)
P4a `UNION` (distinct) distributes	**0**	
P4b `INTERSECT` distributes	**0**	
P4c `EXCEPT` distributes	**0**	

2.4 Injection strategies

Strategy A — wrap and filter the projected column.

```
SELECT * FROM ( <the whole set-op body> ) AS _smelt_inc
WHERE event_time >= '<start>' AND event_time < '<end>'
```

This is literally the manual rewrite the current error suggests. It is uniform, provably correct **when `event_time` is projected by every branch**, and it does not care that branch 1 aliased it from `created_at` and branch 2 from `purchased_at`. It is the natural fit for the common per-row case.

- ✓ Covers merging event streams: `web_events UNION ALL mobile_events`.
- ✗ Fails for **aggregating** branches — `SELECT date, SUM(x) … GROUP BY date UNION ALL …` — whose output projects `partition_column` (`date`), not `event_time`. You cannot filter on a column that is not in the output, and filtering *after* aggregation would be wrong. These stay rejected for now.

Strategy B — inject into each branch. Walk the branch chain (`SelectStmt::set_operation_select()`, `ast.rs:1180`) and run the existing single-select injection on each branch's `WHERE`, before each branch's `GROUP BY`. This handles aggregating branches and per-branch differing sources, but each branch's `event_time` must be independently resolvable in that branch's `FROM` scope. Deferred. (Strategy B is a special case of the general "push the proven-safe filter toward the sources" argument developed in **Part 4**.)

2.5 Heterogeneous branches — "one side is not large"

A `UNION ALL` whose branches differ in source characteristics splits into three cases that look alike but are not:

1. **Small side is a genuine, low-volume timeseries source.** Correctness- identical to the symmetric case: the small branch still carries a real `event_time`, so Strategy A filters it exactly like the big side. "Small" only affects *cost*. This composes cleanly with the existing per-source pushdown, which already **skips** sources lacking `timeseries:` (`inject_source_filters`, `transformer.rs:65`) — so a small lookup branch is naturally full-scanned while the partition-scoped write still bounds output.
2. **Small side is a static/lookup with no real `event_time` (the hazard).** A `UNION` forces all branches to share columns, so this branch must still emit *something* in the `event_time` slot — a literal, or `NULL`.
   - A **constant** timestamp lands the branch in exactly one partition, ever.
   - A **`NULL`** makes `NULL >= start` false, so the branch **never contributes to any incremental run**, yet **does** appear in a full refresh. This silently breaks the incremental ≡ full invariant. Property **P3** above reproduces exactly this: 1 violating row, the `NULL`-stamped branch.
3. **Small side is a dimension meant to appear in every partition.** Genuinely incompatible with partition-scoped DELETE+INSERT — a "must appear everywhere" row cannot be reconciled with a run that only rewrites the current window's partitions. This is a **JOIN/broadcast, not a `UNION`**, and is an explicit **non-goal**.

The sharpened eligibility condition. The correct precondition for a branch is *not* "projects `event_time`" (the current outer-select check). It is stronger: **each branch's `event_time` must be a monotone function of that branch's own source event-time — i.e. the branch must itself be independently partitionable.** A branch emitting a constant/`NULL` timestamp is a *static seed*, not a partitionable stream, and needs separate treatment (computed once into one partition), or must be rejected with a message that names the real problem.

2.6 What else must change beyond injection

Even for the narrow Strategy-A slice, deleting the E1 guard is not enough — two other gates assume a single flat SELECT:

- **`incremental::detect`** (`incremental.rs:132+`) runs `analyze_select` over the whole SQL; its A3–A6 checks would misfire on set-op syntax and must run **per branch**.
- **Source-bound derivation** (`analysis/source_bounds.rs`) must consider each branch's sources when deriving pushdown bounds.

2.7 Recommendation for E1

Ship the **provably-safe, common slice** first:

- **Scope:** `UNION ALL` only, where **every branch is independently partitionable** (projects a real, monotone `event_time`).
- **Mechanism:** Strategy A (wrap-and-filter on the projected `event_time`), with A3–A6 lifted to run per branch.
- **Keep rejecting** (for now): aggregating-branch unions, `UNION` (distinct)/`INTERSECT`/`EXCEPT`, and any branch emitting a constant/`NULL` event-time — but replace the "rewrite this by hand" message with an honest "not yet supported" that names the specific branch/limitation.

This unlocks the pattern users actually hit — merging multiple event streams into one timeline — with a bounded, mechanical change whose correctness is both proven (§2.2) and measured (§2.3).

Part 3 — Condition deep-dive: subquery in `FROM` (B4 / E2)

3.1 Why it is rejected

A derived-table subquery — `SELECT … FROM (SELECT …) AS t` — is rejected by **two independent gates**, with different sharpness and different escape hatches:

- **B4 — planner safety, blunt, overridable** (`incremental.rs:273`). This is a *textual* test: if `analysis.from_text` contains a `(` and does not contain `smelt.ref(` / `smelt.source(`, the model is refused with "subqueries in FROM clause are not yet supported." It carries the `allow_subqueries` override (Pathway A, B-group), so `--allow-downgrade` or a per-model `safety_overrides.allow_subqueries` turns it off.
- **E2 — event-time injectability, sharp, *not* overridable** (`rule_diagnostics.rs:200`). This one parses. When the outer FROM's table expression starts with `(`, it extracts the inner SQL (`extract_balanced_parens`) and asks whether the inner SELECT projects `event_time_column` (`is_column_projected_in_sql`, honouring `*` and aliases). If not, it emits an **`Error` diagnostic** — surfaced in the editor via `smelt-db` and enforced at the runtime pre-execute gate. `--allow-downgrade` does **not** bypass it.

So a subquery in `FROM` is doubly blocked, and the two gates disagree about *what* the problem is. B4 says "subqueries, categorically, not yet." E2 says "subqueries that hide `event_time` from the outer scope." E2 is the real correctness statement; B4 is an older, coarser guard that predates it. The tension matters: even a subquery that *does* project `event_time` (so E2 is satisfied) is still refused by B4 unless the user reaches for the override.

Neither rejection is a statement that subqueries are semantically incompatible with incremental refresh. As with `UNION` (§2.1), the question is whether the time-window predicate can be applied at a point where it is (a) *visible* and (b) *equivalent to filtering the underlying source*.

3.2 Injection is already mechanically fine — the real question is pushdown validity

This is the crucial contrast with the `UNION` case. For `UNION ALL`, the blocker was the **injection point**: a trailing `WHERE` binds to the last branch only (§2.1). For a subquery in `FROM`, the injection point is **already correct**. `inject_time_filter` (`transformer.rs:272`) appends the predicate to the **outer** SELECT's `WHERE` (or, absent one, after the outer `FROM`). The outer `FROM` is `(Q) AS t`; the injected `WHERE t.event_time >= … AND < …` sits *after* the closing paren and references the column the subquery projects. Mechanically, nothing is mis-scoped.

The real question is **semantic**: is filtering the *output* of the subquery `Q` by `event_time` equivalent to what a full refresh computes? That holds exactly when selection on `event_time` **commutes** with everything `Q` does:

```
σ_event_time( Q(R) )  =  Q( σ_event_time(R) )
```

So the subquery deep-dive is not an injection-plumbing problem (as `UNION` was); it is a **predicate-pushdown-through-a-relational-operator** problem. The answer depends entirely on what `Q` contains:

Body of `Q`	Commutes with `σ_event_time`?	Verdict
project / rename / row filter only (transparent)	yes — selection pushes through freely	**safe**
JOIN where `event_time` comes from the driving (fact) side, dimensions are lookups	yes — filtering the fact side is unaffected by the lookup join	safe (same story as flat-model join pushdown)
aggregation whose `GROUP BY` key ⊇ `partition_column`, projecting the key as `event_time`	yes — each group lives in exactly one window	safe, but `event_time` here *is* the partition key (see §3.3)
window function whose frame crosses windows, `DISTINCT`, `LIMIT`, `ORDER BY+LIMIT`	**no** — these do not commute with a row predicate	unsafe — same reasons as the flat-model B1/B6/B3 rejections, one level down

The transparent row — "I wrapped my query in a subquery for readability / to alias a computed `event_time`" — is the overwhelmingly common case, and it is provably safe. The unsafe cases are unsafe for reasons smelt *already* names at the top level; nesting them inside a derived table does not change the algebra, only where it is written.

3.3 The CTE inconsistency (the sharpest finding)

The current E1/E2 error message tells the user to "rewrite as a CTE or subquery that projects `event_time` through all branches." Follow that advice literally and something revealing happens.

A CTE-form query —

```
WITH t AS (SELECT …, created_at AS event_time FROM smelt.silver.events)
SELECT * FROM t
```

— has an **outer `FROM t`**, which contains **no parenthesis**. Therefore:

- **B4 passes**: `from_text` (`"FROM t"`) contains no `(`.
- **E2 passes**: the outer table expression does not start with `(`, so the subquery-projection check never runs.

The *semantically identical* derived-table form —

```
SELECT * FROM (SELECT …, created_at AS event_time FROM smelt.silver.events) AS t
```

— is rejected by both. In SQL a CTE reference and a derived table denote the same relation; a user can convert one to the other by rote. So today's gate keys on **syntax, not semantics**, with two consequences:

1. **The subquery rejection is largely theatre.** Any user who hits it can mechanically switch to a `WITH` and get the exact same execution — the CTE path already does what the subquery path refuses to do. We are rejecting a pattern we simultaneously allow by another spelling.
2. **If the pattern were genuinely unsafe, the CTE path would be a silent soundness hole.** The commutation question in §3.2 is identical for the CTE and the derived table. If an aggregating / windowed / `DISTINCT` body is unsafe to time-filter, the CTE form ships that unsafe query *with no gate at all* (its only backstop is E2's set-operation check and whatever the outer SQL fails to compile). Whatever we decide for subqueries must be decided for CTEs in the same breath — they are one problem.

This is the strongest argument for replacing B4+E2 with a single **body-structure** check applied uniformly to derived tables **and** CTE bodies: classify `Q` as transparent / aggregating-aligned / order-sensitive per §3.2 and gate on *that*, not on whether a paren appears after `FROM`.

3.4 Source-bound pushdown already reaches into nested bodies

One half of incremental compilation already handles subqueries correctly today. `inject_source_filters` (`transformer.rs:65`) wraps each `smelt.<path>` reference in a pushdown subquery `(SELECT * FROM smelt.<path> WHERE partition_col …)` by **textual replacement of the ref token**. Because it matches the ref wherever it appears in the SQL string, it descends into a derived table or a CTE body without caring about nesting depth — a `smelt.ref` inside `FROM (SELECT … FROM smelt.silver.events)` gets its per-source bound just as a top-level ref would.

So the *cost-optimisation* half of incrementalisation is already nesting-agnostic. Only the *correctness window-filter* half (the outer `event_time` predicate) rejects subqueries — and, per §3.2, it rejects them for a reason that only actually applies to non-transparent bodies.

3.5 Empirical confirmation (DuckDB v1.4.4)

Harness: `docs/research/harness/20260701-subquery_incremental.sql` (run with `duckdb -box < …`). As in §2.3, each property reports violating rows via `|(L EXCEPT ALL R) ⊎ (R EXCEPT ALL L)|`; `0` = the identity holds. Q1 and Q4 double as the Part 4 push-to-source check — their right-hand side pushes the filter to the source (below the projection / below the aggregate) and matches the outer-clamp left-hand side exactly.

Property	Violations	Meaning
Q1 transparent body: σ_e(π σ' R) = π σ'(σ_e(R))	**0**	selection pushes through project/filter to the source (§4.3 row 1)
Q2 two adjacent windows over a transparent subquery = full refresh	**0**	incremental ≡ full for the safe slice
Q3 derived table vs. equivalent CTE produce identical result	**0**	confirms §3.3 — the two spellings are one query
Q4 aggregating body, `GROUP BY created_at ⊇ partition_column`	**0**	group-aligned aggregation pushes below the aggregate (§4.3 row 2)
Q5a `LIMIT` body — outer clamp vs. pushed	**30**	`LIMIT` does not commute with the window predicate (hazard reproduced)

Property	Violations	Meaning
Q5b cross-window frame (`SUM() OVER (ORDER BY …)`) — outer clamp vs. pushed	**800**	an unbounded frame depends on out-of-window rows; not naively pushable (hazard reproduced)

The two hazards (Q5a/Q5b) are exactly the non-commuting bodies §3.2 flags as unsafe: their non-zero counts confirm the pushdown wall and the eligibility wall coincide (§4.3) — where the filter *cannot* be pushed is precisely where the model must *not* be silently incrementalised.

3.6 What else must change beyond the gate

As with §2.6, lifting the rejection for the safe slice touches more than the two guards:

- **B4 and E2 collapse into one body-structure classifier.** Replace the text-`contains('(')` test and the paren-prefix test with a parse-based check that (a) resolves the outer `event_time` to a subquery/CTE-projected column and
  (b) classifies the body as transparent / group-aligned / order-sensitive. Apply it to CTE bodies too (§3.3), closing the current CTE bypass.
- **`incremental::detect`'s A3–A6** (`incremental.rs:132+`) run `analyze_select` over the outer query; for a derived table the `partition_column` / `unique_key` / `event_time` checks must resolve against the **subquery's** SELECT list, not the outer one that just says `SELECT *`.
- **Bound derivation** (`analysis/source_bounds.rs`) already descends textually (§3.4); confirm it still attributes bounds to the correct source when the ref is nested, and that the outer `event_time` alias is traced back to a real source partition for the window filter.

3.7 Recommendation for B4 / E2

Ship the **transparent-body slice** first, and unify the syntax split:

- **Scope:** a single derived-table subquery (or CTE body) whose body is *transparent* — projection, renaming, and row filters only, projecting a real, monotone `event_time`. This is the "wrapped for readability / to alias `event_time`" case users actually write.
- **Mechanism:** no change to `inject_time_filter` — the outer-SELECT injection is already correct (§3.2). Replace B4's text test and E2's paren-prefix test with one parse-based body classifier, applied identically to derived tables and CTE bodies (§3.3).
- **Keep rejecting** (for now): bodies containing aggregation not aligned to the partition key, cross-window window frames, `DISTINCT`, or `LIMIT` — but with an honest message that names the offending construct inside the body, not a blanket "subqueries not yet supported." Crucially, apply the **same** rejection to the CTE spelling, so the two forms stop disagreeing.

Decision (2026-07-01): unify on semantics, not syntax. B4 and E2 are replaced by one parse-based body classifier that resolves the outer `event_time` to a real source column and classifies the intervening operators, applied identically to derived tables and CTE bodies. The syntactic paren-test is retired. Where the proven-safe filter should then be *injected* is a cross-cutting question in its own right — see **Part 4**.

Part 4 — Cross-cutting: where should the proven-safe filter be injected?

This section is not about *one* condition. It applies to every relaxation in this document — `UNION` branches (§2.4 Strategy B), subquery/CTE bodies (§3.2), and plain sources alike — because they all raise the same follow-on question: once we have *proven* a window predicate is safe, **at what depth in the query do we write it?**

4.1 The prompt: proving safety and licensing pushdown are the same fact

The eligibility proof throughout this document is a **commutation** statement: σ_event_time distributes over the operators between the outer select and the source (§2.2 for set ops, §3.2 for subqueries). But commutation is *exactly* the classical precondition for **predicate pushdown** in a query optimiser. So the analysis that decides "is this model incrementalisable?" is the same analysis that decides "how deep can the time filter be pushed toward the scan?" They are one computation, and it is wasteful — and, as §3.3 showed, unsound-prone — to answer the eligibility half while leaving the placement half to chance.

The practical worry the placement question raises: if we prove a wrapped subquery is safe and then inject the filter only on the *outer* select, we are trusting the downstream engine's optimiser to push that predicate back down through the derived-table (or aggregation, or CTE) boundary to the scan. If it doesn't, the model is *correct* but scans the whole source anyway — we did the proof work and handed the engine none of the benefit.

4.2 Today: two layers, two windows

The runtime already injects at two depths, for two different purposes (`execute.rs:895–913`):

1. **Outer output-clamp** — `inject_time_filter` appends `event_time >= filter_start AND < filter_end` to the **outer** select, using the **widened write window** (the run window extended backward by the derived lookback). Its job is *correctness of output*: only the current window's rows may be written.
2. **Per-source scan filter** — `inject_source_filters` wraps each `smelt.<path>` reference in `(SELECT * FROM smelt.<path> WHERE partition_col …)` using the **un-widened run window** plus that source's derived bounds. Its job is *scan pruning*, and because it is a textual ref-replacement it already descends into subquery / CTE bodies (§3.4). Concretely it emits `partition_col >= run_start − before_secs AND < run_end + after_secs` (`transformer.rs:82–84`), and the per-source `before_secs`/`after_secs` come from bound derivation — so the *scan is genuinely widened*; what is "un-widened" is only the run window it starts from, before each source's own margin is added. The scan is not stuck at the bare run window.

So push-to-source is not a new idea in smelt — it already exists as the second layer. What is missing is that the two layers are derived **independently**: the outer clamp uses the widened *write* window (run window + derived lookback), while each source filter starts from the *un-widened* run window and re-derives its own `before_secs`/`after_secs`. Nothing guarantees the two windows agree — and code inspection (2026-07-02) shows the problem is sharper than a possible under-approximation. The two lower bounds come from **two separate analyzers over the same SQL**: `compute_effective_window` (`analysis/temporal.rs`) feeds the write window's `filter_start = run_start − k` (`windowing.rs:179–186`), while `derive_model_bounds` (`analysis/source_bounds.rs`) feeds the scan's `before_secs`; for a model with one `INTERVAL 'k'` both independently derive ≈ k. **Equal is not enough.** The DELETE+INSERT covers the widened write window `[run_start − k, run_end)` — the DELETE is deliberately matched to the INSERT's clamp for idempotency (`execute.rs:925–941`) — so every run *re-writes* the margin rows in `[run_start − k, run_start)`. Whenever the lookback reflects a genuinely cross-window reach (a cross-partition frame admitted via `allow_window_functions`, or a Form-B `WHERE`/join offset), those margin rows' own frames reach a further k back, to `run_start − 2k`; the scan stops at `run_start − k`, so the run recomputes them with clipped frames and **overwrites the previously-correct trailing k of the prior window with understated values** — the W2 under-read (§8.5), by construction, on every run. (For the B1-compliant intra-partition case the frame is truncated at the partition edge, so the rewrite is merely redundant, not wrong.) Covering the rewritten margin would need a scan margin of 2k; the cleaner fix is the Part 8 exact-clamp design, which reads the margin but never re-writes it. Deriving both windows from one downward walk (§4.5) removes the mismatch by construction; today the *correctness* filter is the one left at the outer level, dependent on the engine to prune.

4.3 The unification: eligibility *is* maximal pushdown depth

Reframe the whole audit as a single downward walk. Starting from the outer `event_time` column, push `σ_event_time` toward the sources, one operator at a time, stopping at the first operator it does **not** commute with. The point where it stops is where the filter must be written; how far it got is the eligibility verdict:

Body between outer select and source	σ pushes to…	Verdict
transparent (project / filter / rename), no lookback	the **source scan** — one filter both clamps output and prunes the scan	safe; a single source-level filter is strictly better than the outer wrap
aggregation with `GROUP BY` key ⊇ `partition_column`	just **below the aggregate** — the predicate is on the group key, which is a function of input columns, so it lands on the source too	safe; group-local
window function with a bounded `RANGE` lookback	genuinely **two windows**: a widened scan bound at the source *and* an exact output clamp above the window operator — both layers are load-bearing	safe but irreducibly two-layer
`DISTINCT`, `LIMIT`, cross-window frame, non-monotone `event_time`	**nowhere** — σ does not commute past it	reject — the pushdown wall and the eligibility wall are the same wall

Two things fall out of this table:

- For the **transparent slice** (the common subquery/CTE and single-stream `UNION ALL` cases), there is no lookback, so the output-clamp window and the scan window **coincide**. The two layers of §4.2 collapse into one filter, written at the source. Pushing to source is not just an optimisation here — it is the *simpler* mechanism.
- The cases that *require* the two-layer split are exactly the ones with a lookback margin (window functions), where the scan window is deliberately wider than the output window. There the outer clamp is irreducible. This tells us **when** two layers are warranted (a real lookback) versus when the second layer is just belt-and-suspenders (no lookback → the outer clamp is redundant with a source filter on the same window).

4.4 Why push at compile time rather than trust the engine

smelt's stated identity is a **compiler and orchestrator, not a query engine** (root `CLAUDE.md`), and this is where that identity pays off:

- **Partition pruning needs the predicate *at the scan*, on the *partition column*.** On a partitioned store (Databricks/Spark Delta, Hive-partitioned Parquet), the engine prunes files only when the filter reaches the scan on the partitioning column. A predicate stranded above an aggregation or a derived table will be applied *after* a full read — correct, but unpruned.
- **Optimiser pushdown through derived tables/aggregations is not guaranteed and differs by backend.** smelt targets multiple engines; relying on each one's optimiser to rediscover a pushdown we already proved safe makes the scan cost a function of the backend rather than of the plan. Pushing it ourselves makes the bound *guaranteed* and *portable*.
- **We already did the proof.** The commutation argument that lets us incrementalise is precisely the license to relocate the predicate. Emitting it at the source is free correctness-wise and turns the proof into an actual scan reduction.

This is the same argument as §2.4's Strategy B (inject into each `UNION` branch) generalised: whenever we can prove σ commutes down to the sources, we should emit the filter *there*, and treat the outer clamp as needed only when a lookback margin makes the scan window legitimately wider than the output window.

4.5 What this implies for the work

- **Fold placement into the classifier.** The body classifier that replaces B4/E2 (§3.7) should not just return safe/unsafe — it should return the **deepest injection point** for `event_time` (source scan, below-aggregate, or above-window-with-lookback). Injection then writes the filter there.
- **Let the source filter subsume the outer clamp when there is no lookback.** For the transparent slice, skip the outer `inject_time_filter` wrap and rely on a source-level filter on the exact run window — fewer moving parts, guaranteed pruning. Keep both layers only when a derived lookback makes them genuinely distinct windows.
- **Unify the two bound derivations.** Today the output-clamp window and the per-source bound are computed by different code with different windows (`execute.rs:895` vs `:913`). If placement is one downward walk, the windows should be derived once, per source, from that walk. This is not just cleanup: the two windows being derived independently is the latent under-read risk of §4.2 (a source margin that under-covers the outer write window), which a single per-source derivation eliminates by construction.

4.6 Open risk

Pushing the filter to the source changes *which* rows the inner query sees, which is sound only if the pushed predicate is genuinely equivalent — the whole point of the commutation proof. The danger is a body the classifier mis-labels as transparent (e.g. a scalar subquery in the SELECT list that secretly depends on unfiltered rows, or a non-monotone `event_time` expression). The classifier must be **conservative**: when it cannot prove the outer `event_time` traces back monotonically to the source partition column, it stays at the outer clamp (today's behaviour) or rejects — it must never push a filter it has not licensed. This is the same conservatism the empirical harness (§3.5, Q5) exists to keep honest.

4.7 Composing with a user-authored event-time filter

A model may already carry its own range predicate on the event-time — a hard floor the modeller wants on *every* run (`WHERE event_time >= DATE '2020-01-01'`), or a business rule that references it (`WHERE event_time < order_ts`). Two questions arise; both fall out of the machinery already built.

(a) A user `WHERE` on the same event-time column composes by intersection. The injected window predicate is ANDed onto whatever the outer `WHERE` already holds, so the effective scan is the *intersection* of the user's range and the run window. This is correct, and — crucially — incremental ≡ full is preserved *because the same user predicate constrains both the windowed rebuild and the full-refresh oracle*. A floor `event_time >= '2020-01-01'` narrows both identically; a per-window rebuild of `[t_i, t_{i+1})` intersected with the floor still reassembles to the full-range result intersected with the floor. A pre-existing event-time range predicate therefore never *breaks* incrementalisation. The only open choice is pushdown depth (Part 4): a user predicate on the *monotone-traceable* event-time can be pushed to the source alongside the injected filter (they are conjunctive range predicates on the same column), tightening the scan further; a user predicate the classifier cannot trace (e.g. `event_time < order_ts`, a two-column comparison) stays at the outer clamp — not a hazard, merely un-pushable, exactly the §4.6 fallback.

(b) A filter written against the *output* (post-transform) clock still resolves via the trace. When the model projects `event_time = f(source_col)` for a monotone `f` (§6.2) and the user filters on that projected `event_time`, the Part 6 trace is what rewrites the predicate onto `source_col` for

pushdown — the same `project(predicate)` move Iceberg makes (§7.4). So "the `WHERE` references the downstream clock, not the raw source column" is not a barrier to pushdown; it is the exact situation the monotonicity primitive exists to handle. Absent a traceable transform, the predicate is applied at the outer clamp only, and correctness still holds — it simply does not prune.

The general rule: a pre-existing event-time range predicate is always *safe* (it applies equally to the incremental sequence and the oracle); at worst it fails to push and stays above the scan.

Part 5 — Condition deep-dive: joins (the un-gated construct)

Every condition worked so far is a **rejection** the audit asks us to relax. Joins are the opposite shape: a base-relational construct that is **never rejected** for incremental eligibility, yet is not universally safe. So the deep-dive runs the same four-step frame in reverse — *why is it allowed → is the allowance a correctness law or an accident → where is it actually unsafe → what gate makes the safe slice safe*.

5.1 The asymmetry: audited nowhere, rejected nowhere

A join reaches smelt through two surfaces, and neither is gated:

- **Inline SQL** — `FROM fact JOIN dim ON …` written directly in the model. `analyze_select` (`analysis/mod.rs:36`) captures the `FROM` clause as **opaque text** (`from_text`, `mod.rs:86`); it never inspects join structure. So a join model sails through every existing check: A2 (parses), A3–A6 (run on the `SELECT` list and `GROUP BY`, not the join), B4 (no `(` in `from_text` unless a subquery is also present), E1 (`has_set_operation` false), E2 (outer `FROM` does not start with `(`). Nothing looks at the join.
- **Declared `joins:` frontmatter** — a first-class feature (`JoinSpec`, `logical.rs:97`) describing side-joined dimensions with a **declared cardinality** (`Cardinality::{OneToOne, OneToMany}`, `logical.rs:132`) that the planner already trusts for join elimination (`EliminateUnusedLeftJoin`; the §2OE declared-cardinality soundness caveat). Crucially, `joins:` is **annotation, not construction**: `planner_integration.md` §7 requires each entry's `table` to *already appear as a join alias in the body's outermost FROM* (enforced by the `JoinsMismatch` diagnostic), its sole planner consumer is join *elimination*, and it is gated behind `unstable_schema: true`. So the declaration never injects a join into the query — it only describes, for optimisation, a join the author already wrote in SQL. Two consequences for this audit: the actual join always lives in the inline `FROM`, so the incremental gate must read the SQL and cannot rely on the frontmatter being present; and the declared cardinality is a *safety signal* the gate could reuse, but nothing wires it to *incremental* eligibility today.

This is the §3.3 pattern (a query allowed by one spelling while another is refused) taken to its limit: joins are allowed by **both** spellings, with **no** gate in either. The CTE bypass was a hole because the derived-table form was gated and the CTE form was not; the join hole is larger because *neither* form is gated at all, even though — unlike a transparent subquery — a join is not uniformly safe to time-window.

5.2 What happens today — two injections, one of them unsound

Trace an incremental join model through the two filter layers of §4.2:

1. **Outer output-clamp** (`inject_time_filter`, `transformer.rs:272`) appends `event_time >= start AND < end` to the outer `WHERE`. The column name is emitted **unqualified** (`transformer.rs` builds the predicate from the raw `event_time_column` string). On a join this has two failure modes: if the name is **ambiguous** across both sides, the engine raises a bind error — *loud*, which is tolerable; if it resolves to the **dimension** side, the model filters on a lookup's timestamp rather than the fact stream's clock — *silent*

and wrong. The A5 guard (`incremental.rs:195`) does not catch either: it is a bare `stripped_sql.contains(event_time_column)` substring test with no scope or qualification awareness.

2. **Per-source scan filter** (`inject_source_filters`, `transformer.rs:65`) wraps **each** bounded `smelt.<path>` reference independently in `(SELECT * FROM smelt.<path> WHERE partition_col >= run_start AND < run_end)`. Sources without a derived bound (no `timeseries:`) are left untouched (`source_bounds` only emits bounds for timeseries refs). So:
   - a **static lookup** dimension (no `timeseries:`) is correctly full-scanned;
   - but a **timeseries dimension used as a lookup** *does* get a bound, so it is silently time-windowed on its own partition column — dropping exactly the dimension rows outside the current window that the join needs. **This is a soundness bug in an already-allowed pattern**, not a missing feature.

5.3 Correctness taxonomy of join shapes

The safety of time-windowing a join turns on **which input carries the model's event-time clock** and whether every other input is invariant to the window:

Join shape	event_time source	Other input treated as	Window-filter safe?	Verdict
fact JOIN **static** dim (no `timeseries:`)	fact	full-scanned lookup (untouched)	yes — σ_e(F JOIN D) = σ_e(F) JOIN D	**safe** (same story as flat-model join pushdown, §3.2 row 2)
fact JOIN **timeseries** dim used as lookup	fact	independently windowed source (bug)	**no** — the dim's own source filter drops rows the join needs	**hazard — silent today**
fact JOIN fact, joined on a **non-partition** key	one or both	independently windowed source	**no in general** — a fact row in window *W* may need a counterpart outside *W*	**hazard — silent today**
fact JOIN fact, joined **on the partition key**	both, aligned	co-windowed	yes — both sides share the window boundary	safe (narrow)
fact JOIN **fact**, equi-key **plus a bounded time band** (`parent.ts BETWEEN child.ts − k AND child.ts`)	the child (driving) fact	second fact read only within a finite band	yes — **with a derived lookback k**	safe (interval join, see below)
dim-side `event_time`	dimension		ambiguous — a lookup's timestamp is not the stream clock	reject / at minimum loud
OneToMany fan-out	fact	row-multiplying	needs care — fan-out interacts with `unique_key` MERGE	reject unless reconciled

The declared `joins:` **cardinality** is precisely the discriminator between the safe rows (1:1 / lookup dimension, elidable, window-invariant) and the unsafe ones (1:N fan-out, or a second clock-bearing fact). smelt already trusts this declaration for join *elimination*; the same declaration should license — or refuse — time-filter pushdown to the fact side only.

The interval/temporal join is a fourth, incrementalisable shape — and the one most likely to matter in practice. Joining a child event to its *parent* on an equi-key **plus a bounded time band** — `parent_id` **and** `parent.ts BETWEEN child.ts - k AND child.ts` (equivalently `child.ts BETWEEN parent.ts AND parent.ts + k`), the "attach the most-recent parent state to each child event" pattern — is *not* the unbounded multi-clock hazard of J4. The band gives the second fact a **finite lookback k**: the child fact is the driving clock, and the parent fact need only be scanned over `[run_start - k, run_end)` rather than full or (unsoundly) windowed on its own clock. This is structurally identical to the Part 8 bounded-`RANGE` window result — a *widened scan* on the non-driving input plus an *exact output clamp* on the driving one — and it is exactly what Flink classifies as an **append-only interval join** (§7.2, "interval/temporal = append"), as opposed to a *regular* (unbounded) equi-join which is *updating*. The safety hinge is the band: an equi-key join carrying **no** time band is the J4 multi-clock hazard (the parent counterpart can sit arbitrarily far outside the window); adding a band whose width `k` is a compile-time constant is precisely what makes the second clock's reach finite and therefore derivable. So the additional join key the pattern needs — a `parent_id` equi-predicate *alongside* the time band — is allowed and expected; what the eligibility test keys on is not the number of keys but whether **exactly one** of them is a bounded temporal band against the driving clock.

5.4 The sharpened eligibility condition

A join is incrementalisable exactly when:

1. **One input is the driving fact** — it carries the model's `event_time`, which traces monotonically back to that source's partition column (the same "independently partitionable / monotone event-time" primitive §2.5 and §4.6 require, and which does not yet exist in `smelt-logical`); and
2. **Every other input is a window-invariant lookup** — its contribution to any output row is independent of which window is being built. A declared 1:1 (or 1:N-lookup) dimension with no timeseries clock qualifies; a second timeseries fact joined on anything other than the shared partition key does not.

Condition 2 has a **bounded-lookback relaxation** (the interval/temporal join, §5.3): a second fact joined on an equi-key *and* a time band of compile-time width `k` is not window-invariant, but its read is confined to `[W.lo - k, W.hi)`, so it is incrementalisable with a Part-8-style widened scan rather than a full one. The uniform test across all three shapes is whether the non-driving input's contribution to window `W` is confined to `[W.lo - k, W.hi)` for a static `k` — invariant lookups have `k = 0` (full-scanned but window-independent), band joins a finite `k > 0`, and an unbounded second clock (J4) no finite `k` at all.

Under that condition the correct injection (per Part 4) is: push the window filter to the **driving fact's scan only**, and leave every lookup full-scanned. That is *almost* what the runtime does today for un-timeseries'd dimensions — the gap is that `inject_source_filters` wrongly also windows a *timeseries* lookup (§5.2), and that nothing identifies which single input is the driving fact.

5.5 Empirical confirmation (DuckDB v1.4.4)

Harness: `docs/research/harness/20260701-join_incremental.sql` (run with `duckdb -box < …`). As in §2.3/§3.5, each property reports violating rows via `|(L EXCEPT ALL R) ⊎ (R EXCEPT ALL L)|`; `0` = the identity holds. J1/J2 confirm the safe slice; J3–J5 reproduce the three hazards of §5.3 with a `dim_ts` whose 50 users all registered on Jan 1, well before the Jan 3–6 events that reference them.

Property	Violations	Meaning
J1 `σ_e(F JOIN D_static) = σ_e(F) JOIN D_static`	**0**	fact-side filter commutes past a static lookup (§5.3 row 1)
J2 two adjacent windows over `F JOIN D_static` = full refresh	**0**	incremental ≡ full for the safe slice
J3 `F JOIN D_ts` with the dim independently windowed	**400**	silent hazard of §5.2 — windowing the dim on its own clock drops every early registration row, and with it all 400 in-window event rows
J4 `F1 JOIN F2` on a non-partition key, both windowed	**4000**	multi-clock join — windowing both facts drops cross-window counterparts (the fan-out of matched pairs inflates the count)
J5 OneToMany fan-out breaks `unique_key`	**400**	all 400 in-window `event_id`s recur after a 1:N join, so the MERGE key is no longer unique

The three non-zero counts confirm §5.3: the safe slice (J1/J2) is a fact-side filter past a window-invariant lookup, and every shape that puts a **second clock** (J3/J4) or a **row-multiplying join** (J5) between the fact and the output breaks the incremental ≡ full invariant. J3 is the one that fires on a pattern smelt **builds today** — the timeseries-dimension-as-lookup misfilter of §5.2.

5.6 What else must change beyond a gate

- **A real join detector.** `analyze_select` must stop treating `from_text` as opaque and expose join structure: the join inputs, which side each projected column (especially `event_time`) resolves to, and the join keys.
- **Qualified event-time resolution.** Replace A5's substring test (`incremental.rs:195`) with a check that resolves `event_time_column` to a *single* input and confirms it is the driving fact — turning today's silent dim-side misfilter into a named error.
- **Fact-only source filtering.** `inject_source_filters` (`transformer.rs:65`) must window only the driving fact, not every bounded source. A timeseries dimension used as a lookup must be recognised as a lookup and left full-scanned (or its join must be rejected). This is the concrete fix for the §5.2 bug.

5.7 Recommendation for joins

The design goal is a robust, legible eligibility model — not a patch. The three pieces below are the shape a correct implementation takes; they are independent of whether the J3 misfilter is hit by any model in flight today (smelt is early-stage — the point is that the eventual gate reasons about joins correctly, not that a live bug needs stopping).

1. **Make "which input is the driving fact" explicit in the model.** The root cause of the J3/J4 hazards is that `inject_source_filters` treats *every* bounded source as windowable, with no notion of a single clock-bearing input. The eligibility check must resolve `event_time` to exactly one input, window only that input's scan, and full-scan every other input (§5.4). A

timeseries dimension used as a lookup is then correctly full-scanned — the J3 result drops to 0 by construction.

2. **Ship the proven-safe slice (J1/J2):** `fact JOIN (declared 1:1 / lookup) dimension(s)`, `event_time` resolved to the fact side, per-source filter on the fact only, lookups full-scanned. This is the pattern most star-schema marts actually use, and it is exactly the safe row of the §5.5 table.
3. **Reject the unsafe shapes loudly, by name:** multi-clock `fact JOIN fact` joins on a non-partition key (J4), dim-side `event_time`, and OneToMany fan-out without a `unique_key` reconciliation (J5). The message must name *which input carries the clock* and *why the other cannot be windowed*, not a blanket "joins not supported." Legibility here is the usability win — a user who wrote a two-fact join should be told which fact smelt treats as the stream and how to express the other as a lookup.

This slots directly into Part 4: identifying the driving fact **is** the downward `σ_event_time` push — for a join, σ commutes down to the fact scan and stops at the join for every non-fact input. The join deep-dive is therefore not a new mechanism but the join-shaped instance of the same commutation walk, blocked on the same missing monotonicity primitive.

Part 6 — The monotonicity primitive

Three of the worked conditions converge on **one missing analysis**, and this part is its full treatment: §2.5 (`UNION` branches), §4.6 (subquery/CTE pushdown), and §5.4 (joins) all block on the same predicate, and none of the analysis it needs exists in `smelt-logical` today.

The primitive answers a single question about a model's projected event-time:

> *Does this `event_time` expression trace back, monotonically, to a real source partition column — and if so, to **which** column, on **which** source, under **what** constant offset?*

The rest of this part pins that down formally (6.1), classifies what is decidable statically (6.2), says where a static decision is impossible and a declaration is required instead (6.3), shows how the three consumers call the one interface (6.4), proposes its placement and shape in `smelt-logical` (6.5), enumerates the edge cases and the conservative-fallback contract (6.6), and closes with the open questions it raises (6.7). The framing is corroborated in detail by prior art — production optimisers implement exactly this analysis, and the theory names its limits — collected in **Part 7** (see especially §7.4).

6.1 Precise definition

Let a source `S` carry a partition column `p` (its `timeseries.partition_column`) and let the model project an `event_time` value through some expression `e = f(...)`. The runtime uses `event_time` in two places (§4.2): the **outer output-clamp** filters rows on the projected `e` directly (`inject_time_filter`, `transformer.rs:272`), and the **per-source scan filter** filters `S` on its partition column `p` (`inject_source_filters`, `transformer.rs:65`, using the `source_partition_col` carried by `BoundResult::Bounded`, `source_bounds.rs:79`).

Incrementalisation is correct only when these two filters select **the same rows** — i.e. when filtering the output on `e` is *equivalent* to filtering the source on `p`. That is a statement about the function `f` relating `p` (or the source's own event clock) to `e`.

The exact property needed is not order-preservation in the strict sense, nor value-injectivity. It is:

> **Interval-preimage-is-an-interval.** For every window `[lo, hi)` on `e`, the set of source rows with `f(p) ∈ [lo, hi)` is exactly the set with `p ∈ [a, b)` for some thresholds `a, b` (i.e. the preimage of a half-line is a half-line). Equivalently, `f` is **monotone non-decreasing**.

Two clarifications this framing forces, both of which matter to the whitelist in 6.2:

- **Non-decreasing suffices; strict monotonicity is not required.** `DATE_TRUNC` and `CAST(ts AS DATE)` are *many-to-one* (a whole day of timestamps maps to one date) yet still push cleanly, because the model's window boundaries are themselves granularity-aligned: `partition_column` **is** `DATE_TRUNC('day', e)` in the canonical model (`incremental.rs:172–176` reads the partition column's expression `text`). A plateau of f never straddles a window boundary, so the half-line preimage is exact. Requiring strict monotonicity would needlessly reject the single most common shape. (This is exactly the "weakly monotone → push a **closed** source range covering the truncation bucket" rule the ClickHouse/Iceberg/Delta implementations use, §7.4.)
- **We need window-preserving, not value-preserving.** The output-clamp already filters on e verbatim, so it is trivially correct whenever e is projected (that is all E2's `is_column_projected_in_sql` check, `rule_diagnostics.rs:236`, verifies today). Monotonicity is the *extra* fact required to **relocate** that filter onto p at the source — i.e. it licenses the Part 4 pushdown, not the bare injection. This is why the primitive is a prerequisite for the *pushdown* half of every relaxation, and why §4.6 phrases its conservative fallback as "stay at the outer clamp" — the outer clamp needs no monotonicity, only the push does.

There is a second, weaker use where monotonicity still matters even **without** pushdown: the §2.5 *independent-partitionability* / NULL hazard. A `UNION` branch that stamps `event_time` with a constant or `NULL` is a *static seed*, not a monotone image of any clock — it lands in one partition forever (constant) or never passes `e >= start` at all (`NULL`), silently breaking incremental ≡ full (property **P3**, §2.3, 1 violating row). So the predicate has to reject constant/NULL/plateau-collapsing expressions too; "monotone image of a real source clock" is precisely the condition that excludes them. The two uses share one predicate: *e is a monotone non-decreasing, total, source-traceable image of S's clock.*

6.2 What is decidable statically from the SELECT expression

`smelt-parser` already exposes a rich typed expression tree — `Expr` offers `as_column_ref`, `as_function_call`, `as_cast`, `as_extract`, `as_case`, `as_binary` (`ast.rs:1860–1968`), `FunctionCall::name/arguments` (`ast.rs:2240,:2316`), `BinaryExpr::left/right/operator` (`ast.rs:2103–2113`), `CastExpr::expression` and its target type (`ast.rs:2725`). So a real structural classifier is feasible; it does **not** need to be a substring heuristic like the A5 test (`stripped_sql.contains(event_time_col` `incremental.rs:196`). The one plumbing wrinkle: `analyze_select` currently keeps only the *raw text* of each select item (`SelectItemKind::{GroupByKey,…}.text`, `analysis/mod.rs:9–16`) and discards the `Expr` node, so the primitive must either re-parse the event-time expression text or `analyze_select` must be extended to retain the node (see 6.5).

Classify the event-time expression e by walking it from the projected column toward the leaves. The proposed **monotone whitelist** — each form provably non-decreasing across DuckDB/Spark/Postgres, and each independently present in the ClickHouse/Iceberg/Delta whitelists (§7.4):

Form	Example	Monotone?	Traces to
transparent alias / bare column	`created_at AS event_time`	identity	the column, offset 0
qualified column	`f.event_ts AS event_time`	identity	column on the qualified input
`DATE_TRUNC(unit, col)`	`DATE_TRUNC('day', event_ts)`	non-decreasing (step)	`col`
`CAST(col AS DATE/TIMESTAMP)`	`CAST(event_ts AS DATE)`	non-decreasing (truncation)	`col`

Form	Example	Monotone?	Traces to
`date_bin` / `time_bucket` / `FLOOR(col to grid)`	`time_bucket('1 hour', ts)`	non-decreasing	`col`
`col ± INTERVAL '<const>'`	`event_ts + INTERVAL '1 day'`	strictly increasing shift	`col`, offset folds into the bound
`col AT TIME ZONE '<fixed-offset const>'`	`ts AT TIME ZONE 'UTC', … '+05:30'`	strictly increasing constant shift	`col`

This whitelist is exactly why **the output `event_time` need not equal, nor even be named the same as, any input column.** The two transformations users most often reach for are already admitted: **fixed-offset time-zone conversion** (`ts AT TIME ZONE '+10:00'`, a monotone constant shift — named DST zones are *not* monotone; see the blacklist and watch-point (c) below) and **granularity coarsening** (`DATE_TRUNC('month', ts)` / `CAST(ts AS DATE)`, a monotone step). Both trace back to the source clock under a folded offset, so a model that emits `business_month AS event_time` from a daily `created_at` is `Traceable`, not a special case — the primitive handles a *manipulated or renamed* output event-time uniformly. What coarsening additionally introduces — a constraint between the output partition's *granularity* and the run cadence — is **not** a monotonicity question (the transform is monotone either way) and is worked separately in **Part 10**.

The **non-monotone / order-breaking** forms, which must yield *not-traceable*:

Form	Why it breaks	Example
arithmetic on **two** columns	not monotone in either alone; also multi-source (6.6)	`end_ts - start_ts`
`MOD` / `EXTRACT(HOUR/DOW/…)`	periodic — preimage of an interval is a union of intervals	`EXTRACT(HOUR FROM ts)`
`CASE WHEN …`	piecewise; generally neither monotone nor total	`CASE WHEN … THEN a ELSE b END`
`COALESCE(col, <const>)`	injects a constant for `NULL` rows — the §2.5 seed hazard in function form	`COALESCE(event_ts, '1970-01-01')`
`GREATEST/LEAST(col, <const>)`	clamps to a plateau that *can* straddle a window boundary	`GREATEST(ts, '2020-01-01')`
unknown scalar UDF	monotonicity unknowable from the call site (Rice/Richardson, §7.4)	`my_udf(ts)`
constant / `NULL` literal	static seed, not a stream (§2.5 case 2)	`TIMESTAMP '2020-01-01'`, `NULL`
run-nondeterministic clock	`NOW()`/`CURRENT_DATE` shift each run; not source-traceable	`NOW()` (also B5, `incremental.rs:288`)
`CAST(col AS VARCHAR)`	lexical order ≠ temporal order in general	`CAST(ts AS VARCHAR)`
`col AT TIME ZONE '<named DST zone>'`	instant→local wall clock goes **backward** at DST fall-back (…01:59 → 01:01…), so an interval's preimage is a union of two intervals	`ts AT TIME ZONE 'America/New_York'`

Where engine semantics matter. The whitelist is deliberately the intersection of what is monotone on *every* target backend, because smelt is multi-backend (§4.4) and a per-engine monotonicity table would make eligibility a function of the backend rather than of the plan. Two watch-points:

(a) CAST is only whitelisted for date/timestamp target types — `CAST(ts AS VARCHAR)` is monotone *only* for ISO-8601 lexical form and not in general, so it is excluded; (b) month/year `INTERVAL` arithmetic has a non-uniform step but is still monotone non-decreasing, so it is admitted even though the offset cannot be folded to a fixed `Seconds` (it stays a symbolic offset — cf. `source_bounds` approximating `MONTH ≈ 30 days`, `source_bounds.rs:506`); (c) `AT TIME ZONE` is whitelisted **only for fixed-offset zones** (`'UTC'`, `'+05:30'`). For a named zone with DST the instant→local mapping *decreases* by an hour at fall-back — confirmed empirically on DuckDB v1.4.4 (`2024-11-03 America/New_York`: instants one minute apart map to local `01:59` then `01:01`; harness `docs/research/harness/20260702-holistic_aggregate.sql`, property H2) — so the preimage of a local window is a union of two disjoint intervals, not an interval, and an earlier draft's "DST plateaus but never decreases" claim was simply wrong. A future relaxation could admit named zones via a ±1h-widened scan plus an exact output clamp (the Part 8 two-layer move applied to a piecewise-monotone transform, cf. ClickHouse's factor-transformation trick, §7.4), but as a plain whitelist entry it is unsound.

Composition is closed under the whitelist: a composition of monotone non-decreasing functions is monotone non-decreasing, so `DATE_TRUNC('day', CAST(event_ts AS TIMESTAMP) + INTERVAL '2 hours')` traces through all three layers to `event_ts` with a +2h offset. The classifier recurses on the single column-bearing argument at each layer and fails closed the moment a layer has two column-bearing arguments or an unrecognised head. (This is precisely the `preserves_order`-under-composition rule Iceberg enforces and the "a single non-monotone component poisons the chain" caveat the pushdown research draws out, §7.4.)

6.3 Where a static decision is impossible — the declared guarantee

Static classification runs out in three situations: (a) an opaque scalar UDF whose body smelt cannot see; (b) a smelt function (`smelt.functions.*`) whose expanded body is monotone but too large to re-derive cheaply; (c) a genuinely data-dependent monotonicity (e.g. a column the modeller *knows* is append-only-monotone but which the SQL does not prove). For these, the safe default is *not-traceable* (6.6) — but the modeller may supply the guarantee. That the *general* case is undecidable (Rice's theorem for arbitrary functions; Richardson's theorem for elementary expressions, §7.4) is exactly why a declared escape hatch is unavoidable rather than a shortcut.

The natural annotation home already exists and already has this exact "trust the declaration" shape:

- **`FunctionProperties`** (`logical.rs:55–:64`) already carries `deterministic`, `idempotent`, `append_only` — declared, unverified booleans on a smelt function. A `monotone_event_time` (or per-argument `monotone`) property slots in beside them and lets a function-wrapped event-time expression be admitted by declaration when its body is not statically classifiable.
- **`timeseries:` frontmatter** (`config.rs:477`) is where a per-model override would live if the guarantee is about a specific model's projection rather than a reusable function — e.g. asserting that a named event-time expression is monotone in a named source column.

The precedent — and the caution — is the declared `joins:` **`cardinality`** (`JoinSpec`, `logical.rs:103`; `Cardinality::{OneToOne,OneToMany}`, `logical.rs:135`). The planner already trusts that declaration *for optimisation* (join elimination, the §20E soundness caveat). A monotonicity declaration would be trusted **for correctness** — the stakes are strictly higher, exactly as §5.4 and the last open question of Part 5 flag. The design rule that falls out: a declaration may *widen* eligibility, but the conservative static default when no declaration is present must be *reject-the-push*, never *assume-monotone*. This matches the industry posture: every window-forward engine (Spark/Flink/dbt/ SQLMesh/cube.dev, §7.1) takes the monotone-event-time column as a *declaration* and never proves it; smelt's novelty is to *prove* it where it can and fall back to declaration only where it must (§7.5).

6.4 The three consumers call one interface

All three worked conditions reduce to one call with the same signature. The input is a SELECT (or a `UNION` branch), the projected `event_time` expression, and the set of source refs with their declared partition columns (the `BoundContext` already built for bound derivation, `source_bounds.rs:131`; assembled from the graph in `incremental.rs:559–568`). The output is not a bare boolean — per Part 4 it must name the **deepest source column** the filter can be pushed to, so it doubles as the injection-point resolver. (The nearest production analog, ClickHouse's `getMonotonicityForRange`, likewise returns a verdict *struct* — not a boolean — carrying direction and strictness so the caller can rewrite the range correctly, §7.4.)

- **§2.5 `UNION` branches — "independently partitionable".** For each branch, call the primitive on that branch's `event_time` projection against that branch's own sources. A branch that returns *traceable* is a partitionable stream (Strategy A / B is safe on it); a branch that returns *static-seed* is the P3 NULL/constant hazard and must be named and rejected, not silently dropped.
- **§4.6 subquery/CTE conservatism.** Before pushing the proven-safe filter below a derived-table or CTE boundary, call the primitive on the outer `event_time` resolved through the body. *Traceable* → *source-column* licenses the push (Part 4 "eligibility = maximal pushdown depth"); *not-traceable* → stay at the outer clamp (today's behaviour) — never push a filter the primitive did not license.
- **§5.4 joins — "exactly one input carries a monotone event_time".** Call the primitive on the model's `event_time` against every join input. Incrementalisable iff it returns *traceable to exactly one input* (the driving fact); that input's scan is windowed and every other input is full-scanned. Two traceable inputs is the multi-clock hazard (J4); zero is a dim-side or ambiguous clock (reject). This replaces the A5 substring test (`incremental.rs:195`) with a resolution that names *which* input carries the clock.

The shared **output** therefore wants to be, per the Part 4 framing, a *trace* rather than a predicate — the source, the traced source column, and any constant offset — so that `inject_source_filters` can write the filter at that exact column and the offset can be merged into the derived `BoundResult` (whose `source_partition_col`, `source_bounds.rs:79`, is precisely the "deepest source column" the primitive computes). One analysis; three consumers; one injection point. This is structurally the same move as an Iceberg partition-transform `project(predicate)` — rewrite a predicate on the derived value into a predicate on the source column — recovered at compile time from the model's SQL (§7.4).

6.5 Proposed placement and shape in `smelt-logical`

Placement. A new pure module `crates/smelt-logical/src/analysis/monotonicity.rs`, sibling to `source_bounds.rs` and `temporal.rs` under `analysis/`. This respects the **Layered single-ownership** invariant (analysis lives in `smelt-logical`, above `smelt-parser`, below `smelt-db`/`smelt-planner`) and the **Salsa purity** rule (a pure function over parser AST + declared context; any Salsa query in `smelt-db` is a thin wrapper that assembles the inputs and calls it). It has no new dependency — it consumes `smelt-parser`'s `Expr` tree and the existing `BoundContext`.

Shape. A trace enum plus one entry point (illustrative, not final):

```
/// Constant temporal shift folded out of a monotone chain (col ± INTERVAL const).
pub enum Offset { Seconds(Seconds), Symbolic(String) /* e.g. months/years */ }

pub enum EventTimeTrace {
    /// `event_time` is a monotone non-decreasing image of `source_column`
    /// on `source`, shifted by `offset`. The licence to push the filter to
    /// `source.source_column` (Part 4), and to fold `offset` into the bound.
    Traceable { source: String, source_column: String, offset: Offset },
    /// Constant or NULL-injecting — a static seed, not a partitionable stream
    /// (§2.5 case 2 / P3). Names the offending sub-expression.
```

```
    StaticSeed { reason: String },
    /// Cannot prove monotone traceability: non-monotone fn, CASE, multi-source
    /// arithmetic, unknown UDF, run-nondeterministic clock. Conservative — the
    /// consumer must not push (§4.6).
    NotTraceable { reason: String },
}

pub fn trace_event_time(
    event_time_expr: &smelt_parser::Expr,
    ctx: &crate::analysis::source_bounds::BoundContext,
) -> EventTimeTrace;
```

Why this is the natural first implementation phase (as this document claims):

1. **It is the shared blocker.** §2.5, §4.6 and §5.4 cannot ship without it, and they cannot each grow a private, divergent copy without re-introducing exactly the syntax-vs-semantics inconsistency §3.3 exposed. One analysis keeps the three relaxations honest with each other.
2. **It is pure and independently testable.** No injection changes, no runtime changes — a function from `(Expr, BoundContext)` to `EventTimeTrace`. It can be red-green unit-tested on the whitelist/blacklist of 6.2 and property-tested against DuckDB (the §2.3/§3.5/§5.5 harness already reproduces the hazards it must catch: P3, Q5, J3–J5) *before* any consumer is wired up.
3. **Its output type is designed for the consumers, not retrofitted.** Returning a trace (source + column + offset) rather than a boolean means the same result feeds the eligibility verdict *and* the Part 4 pushdown-depth walk *and* the `BoundResult` the runtime already threads — so wiring each consumer is a small follow-on, not a re-analysis.

6.6 Edge cases and the conservative-fallback contract

- **`NULL event_time` (the §2.5 hazard, P3).** Any expression that can evaluate to `NULL` for some rows silently drops those rows from *every* incremental window while a full refresh keeps them. Statically this is decidable for the syntactic cases — a `NULL` literal, or `COALESCE(col, <const>)` — which the classifier routes to `StaticSeed`. It is **not** decidable at this layer for a merely *nullable column* (column nullability is inferred above `smelt-logical`, in `smelt-db`); that gap is called out as an open question (6.7). The conservative stance: a syntactically NULL-injecting form is a seed; a plain column is treated as traceable (matching today's behaviour, which already lets nullable event-times through the outer clamp) and the residual nullability risk is the modeller's, unless we choose to thread nullability in.
- **Constant / static-seed `event_time`.** A literal timestamp → `StaticSeed` (§2.5 case 2). Distinct from a real low-volume stream (§2.5 case 1), which still traces to a genuine clock and is safe.
- **Run-nondeterministic functions.** `NOW()`/`CURRENT_DATE`/`CURRENT_TIMESTAMP` are constant-per-run but *shift between runs*, so they are not source-traceable → `NotTraceable`. This dovetails with the B5 "split the bucket" stub (Part 1 closing list): the monotonicity primitive is exactly the analysis that distinguishes a run-deterministic clock (admissible as an outer clamp, never as a pushed source filter) from a row-nondeterministic one.
- **Multi-source expression.** An `event_time` built from columns of two different sources (e.g. `f.ts` and `d.ts`) has no single source to push to → `NotTraceable`. This *is* the join multi-clock case (§5.4 / J4): the primitive returning "traceable to more than one input" is the same fact as "there is a second clock".
- **The conservative-fallback contract (the load-bearing invariant).** The primitive must be **sound in one direction**: it may return `NotTraceable` for a form that is in fact safe (a false negative — merely a missed optimisation, the consumer stays at the outer clamp), but it must **never** return `Traceable` for a form that is not monotone-source-traceable (a false positive — an unsound pushed filter, the §4.6 danger). Every unrecognised head, every two-column argument, every unknown UDF fails **closed** to `NotTraceable`. This is the same fail-loud / fail-

safe discipline the codebase already enforces elsewhere (`cardinality_from_str` maps any unknown string to the conservative `OneToMany`, `logical.rs:~146`), it is what the empirical harness (P3, Q5, J3–J5) exists to keep honest, and it is the same conservative posture every production optimiser adopts under the undecidability results of §7.4.

6.7 Open questions this raises

- **Column nullability at this layer.** The syntactic NULL forms are catchable, but a nullable source column that produces `NULL` `event_time` rows is not visible in `smelt-logical` (nullability is inferred in `smelt-db`). Do we thread a nullability signal down into the primitive (widening what it can prove), accept the residual risk as the modeller's, or reject any event-time whose leaf column is not provably non-null?
- **Offset folding vs. symbolic offsets.** `col + INTERVAL '1 day'` folds cleanly into a `Seconds` offset that merges with `source_bounds` Form B (`source_bounds.rs:359`). Month/year intervals are monotone but non-uniform — carry them as a `Symbolic` offset the runtime rewrites per-engine, or refuse to push them (outer-clamp only)?
- **Static vs. declared boundary.** How much of the whitelist (6.2) do we ship as static classification before leaning on a declared `monotone` property on `FunctionProperties` / `time-series:` (6.3)? Given the §20E precedent, does trusting a declaration *for correctness* (not just optimisation) warrant a stricter opt-in (e.g. an `unstable_`-style workspace flag, as `provenance:` already requires per `logical.rs:70–73`)?
- **Reusing the trace as the Part 4 injection point.** The trace's (`source, source_column, offset`) is designed to be the "deepest safe injection point." Can `inject_source_filters` / bound derivation consume it directly, or does the operator-by-operator pushdown walk (Part 4 open questions) still need a separate pass for the intervening operators the primitive skipped over?
- **`analyze_select` retaining the `Expr` tree.** The primitive needs the parsed event-time expression, but `SelectAnalysis` currently keeps only raw `text` (`analysis/mod.rs:9`). Extend `analyze_select` to retain the node (one change, many future analyses benefit), or have the primitive re-parse the expression text in isolation (cheaper to land, but re-parses)?
- **Adopt a ClickHouse-style verdict struct?** The prior art (§7.4) returns a four-field verdict (`is_monotonic`, `is_positive`/direction, `is_always_monotonic`, `is_strict`) rather than the three-way enum above. For a *forward-only* event-time we likely only ever need the non-decreasing case, so the enum may suffice — but do we want the direction/strictness fields now to keep the door open for descending clocks and exact endpoint handling?

Part 7 — Prior art and external validation

The audit so far reasons from first principles plus an empirical DuckDB harness (§2.3/§3.5/§5.5). This part checks that reasoning against three external bodies of work: the **academic theory** of incremental computation, the **published eligibility rules** of production incremental-view/materialized-view engines, and the handful of systems that already implement something like the **monotonicity primitive** (Part 6). The headline: every load-bearing claim in this document has independent support, smelt's rejection catalogue (Part 1) is reproduced near-item-for-item by the systems that publish theirs, and smelt's one genuinely novel ambition is to *infer and prove* the monotone-event-time property that every comparable window-forward system instead asks the user to *declare*. Full citations are in **References**.

7.1 Two ways to be incremental — smelt has chosen one

Production systems split cleanly into two camps, and the choice determines whether a monotonicity primitive is even needed:

- **Window-forward over a monotone event-time** (smelt's model): read the next time window and assume the source is append-only/monotone so earlier windows are settled. Shared by **cube.dev** (requires a `time_dimension`), **ClickHouse** MVs (append-only insert blocks), **dbt microbatch** (`event_time`), **SQLMesh** `INCREMENTAL_BY_TIME_RANGE (time_column)`, and — in streaming form — **Spark Structured Streaming** and **Flink** (the watermark *is* the monotone-event-time assertion).
- **Change-tracking / delta-diffing the source** (no monotone column needed): detect *which rows changed* and propagate the delta. Shared by **Snowflake Dynamic Tables** (Stream-style change tracking), **BigQuery** MVs (storage-metadata append diffing), **Databricks Enzyme** (Delta row-tracking + change data feed), and — the theoretical endpoint — **Feldera/DBSP** (Z-sets: every row carries a ± weight, so inserts/updates/deletes propagate uniformly).

The trade is explicit. The window-forward camp needs a monotonicity guarantee (smelt's whole primitive) but is simple and needs no per-source change-tracking metadata; the delta camp sidesteps monotonicity entirely but pays with a whitelist + full-refresh fallbacks (Snowflake/BigQuery) or a stateful differential runtime (Feldera). **smelt is squarely in the window-forward camp, and the monotonicity primitive is the price of admission.** DBSP (§7.3) is the standing proof that the whitelist boundary is a *pragmatic engineering choice*, not a fundamental limit — a fully general engine exists, at the cost of a stateful differential runtime and abandoning the "incremental ≡ full over a window" simplicity smelt is built on.

7.2 The catalogue is externally validated

Every *whitelist* engine that publishes its rules — Snowflake Dynamic Tables, BigQuery MVs, Databricks Enzyme, and (as a changelog "append vs updating" type) Flink — independently reproduces smelt's Part 1 rejection catalogue almost item-for-item. This is strong evidence the catalogue is *correct*, not merely conservative. Databricks Enzyme even ships a named error class, `MATERIALIZED_VIEW_NOT_INCREMENTALIZABLE`, whose sub-conditions map onto smelt's directly: `EXPRESSION_NOT_DETERMINISTIC` (B5), `WINDOW_WITHOUT_PARTITION_BY` (exactly smelt's B1 `PARTITION BY ⊇ partition_column` rule), `AGGREGATE_NOT_TOP_NODE` (the "aggregate must be the outer node" constraint behind §3.2 row 3), and `SUBQUERY_EXPRESSION_NOT_INCREMENTALIZABLE` (B4/E2).

Legend: ✓ incremental-safe · ✗ unsupported → full recompute/reject · ~ conditional · — not separately documented.

smelt rejection	Snowflake Dynamic Tables	BigQuery MV	Databricks Enzyme	Spark Structured Streaming	Flink (changelog)	Materialize	dbt / SQLMesh
`UNION ALL` (E1)	✓	~ preview	✓ (row tracking)	✓ stateless	✓ (least-monotone branch)	✓	trusted
`UNION`/`INTERSECT`/`EXCEPT` (distinct)	✓ `UNION`; ✗ `INTERSECT`/`EXCEPT`	✗	✗ plain `UNION`	✗ (needs dedup)	updating	✓	trusted
subquery-in-`FROM` (B4/E2)	✓ in `FROM`; ✗ outside	✗ ARRAY subq	✗ scalar/expr; `FROM`/CTE ✓	—	✓	✓	trusted

smelt rejection	Snowflake Dynamic Tables	BigQuery MV	Databricks Enzyme	Spark Structured Streaming	Flink (changelog)	Materialize	dbt / SQLMesh
joins (Part 5)	~ OUTER + equality only	~ `INNER`✓, non-leftmost change → full	~ inner/L/R/full ✓; ✗ cross/semi/anti	~ stream=fact; outer needs watermark+range	regular = updating; interval/temporal = append	✓	trusted
`DISTINCT` (B6)	✓	✓ (no exact `COUNT(DISTINCT)`)	✗ plain `DISTINCT`	✗ (`dropDuplicatesWithinWatermark`)	dedup = updating	✓	trusted
window fns (B1/C1)	✓ mostly	✗ all analytic	~ ✓ **only w/ PARTITION BY**	— (event-time `window()`)	`OVER` = append; Top-N = updating	✓	trusted
non-deterministic (B5)	✗ in `SELECT` (✓ in `WHERE`)	✗ `RAND`/`CURRENT_*`	✗ `EXPRESSION_NOT_DETERMINISTIC`	✗	✗	✗	trusted
`HAVING` (B2)	✓	✗ (non-incr only)	✓	—	via updating agg	✓	trusted
`LIMIT` (B3)	✗ `LIMIT/TOP`	—	✗	✗	Top-N = updating	✓	trusted
non-additive agg *(not yet in smelt's list)*	✗ `MEDIAN`/`PERCENTILE_*`	✗ exact `COUNT(DISTINCT)`	—	group-by = updating	retraction state	✓	trusted
verifies ≡ full?	✓ fails `CREATE`	✓ whitelist	✓ algebraic delta or full	✓ engine	✓ engine	✓ engine	✗ **trusts user**

Three observations fall out of the table:

1. **The `UNION`-vs-`UNION ALL` split (§2.2) is industry-standard.** Snowflake ("`UNION` = `UNION ALL` + `SELECT DISTINCT`"), BigQuery, and Enzyme all draw exactly smelt's line — bag-union distributes, distinct-union drags in a `DISTINCT` that does not. smelt's algebraic argument (§2.2) is the same fact these engines encode as a whitelist entry.
2. **An apparent gap that turned out not to transplant: non-additive aggregates.** Snowflake and BigQuery both explicitly exclude `MEDIAN`, `PERCENTILE_CONT/DISC`, and exact `COUNT(DISTINCT)` — they depend on *all* rows, not just the window's. smelt covers `DISTINCT` (B6) but names no such class, and a first pass treated that as a candidate new condition. §9.2 works it and finds the exclusions are artifacts of **delta-style partial-aggregate merging**, which smelt's A4-aligned whole-partition rebuild never performs — in smelt's regime these aggregates are safe, and the classification matters only for `refresh: cumulative`. The general caution this yields: the table validates the catalogue only where the refresh *mechanism* behind a published rule matches smelt's. Corollary: `MIN`/`MAX` are additive-enough that Snowflake *supports* them, but they are non-monotone under *deletes* — merging extrema forward relies on append-only, where a delta engine (Flink) must keep retraction state.

3. **Eligibility vs. cost (from Enzyme).** Databricks decouples "is this incrementalizable?" from "*should* we" — even when incrementalizable, a cost model may still pick full recompute (e.g. large source deletes). This is the reference design if smelt ever wants a "fall back to full-window recompute rather than hard-reject" mode, mapping onto smelt's existing `--allow-downgrade` posture.

7.3 The theory names smelt's safe slice exactly

The empirical safe-slice/hazard split of Parts 2–5 is not a coincidence — it is the classical **monotone / non-monotone frontier** of database theory:

- **Monotone = incrementally-maintainable-without-recompute.** A query is monotone iff $I \subseteq I' \Rightarrow Q(I) \subseteq Q(I')$ (input growth only adds output, never retracts). The **CALM theorem** (Hellerstein's 2010 conjecture; Ameloot–Neven–Van den Bussche's PODS 2011 / JACM 2013 proof) sharpens this to *iff*: monotone queries are **exactly** the add-only, coordination-free class. smelt's window-by-window refresh *is* an add-only computation over a growing event stream, so "incremental ≡ full-refresh" holds by construction precisely for the monotone slice — and *only* there.
- **The operator boundary is the doc's boundary.** Positive relational algebra (σ, π, JOIN, ∪) is monotone; aggregation, EXCEPT/negation, and `DISTINCT`/ `GROUP BY` are non-monotone (a new or late row can retract prior output). This is exactly smelt's safe-slice-vs-hazard split, now grounded in theory rather than the harness.
- **Eligibility proof = pushdown license (one fact).** smelt's commutation statement $\sigma \circ Q = Q \circ \sigma$ (§4.1) is the classical precondition for predicate pushdown (System R 1979; Garcia-Molina–Ullman–Widom §16.2). So "is it incrementalisable?" and "how deep can the filter push?" are one computation — the Part 4 unification is textbook-sound, not a smelt invention.
- **The modern constructive statement: DBSP** (Budiu et al., VLDB 2023 Best Paper). The incremental form of any query is $Q^\Delta = D \circ Q \circ I$, provably equal to recompute. Its cost taxonomy is smelt's taxonomy: **linear** operators (select/project/map/union) satisfy $Q^\Delta = Q$ — the cheap safe slice that commutes with the delta; **bilinear** = joins (product rule); aggregation/negation need nested integ/differentiation. DBSP's linear-vs-bilinear split is the precise algebraic statement of the doc's "safe slice."
- **The streaming analog: watermarks** (Akidau et al., Dataflow, VLDB 2015): a monotone lower bound on event-time completeness that "only moves forward" — the mirror of smelt's bounded-lookback reasoning and the reason late/out-of-order data forces lookback.

The provable limits the design must respect. The invariant cannot be decided in general:

- **Query equivalence for full SQL is undecidable** (Trakhtenbrot, via standard reduction) — so "incremental plan ≡ full-refresh plan" cannot be decided by comparing arbitrary models. Even monotone Datalog has undecidable equivalence: monotonicity buys *maintainability*, not *decidable equivalence*.
- **Monotonicity of an arbitrary function/UDF is undecidable** — Rice's theorem in general; Richardson's theorem (1968) already for elementary real expressions built from $+ \; - \; \times \; \circ$, `sin`, exp, `abs`. So the primitive *cannot* auto-prove monotonicity of an opaque `event_time` expression.
- **Traction is regained only on restricted fragments:** conjunctive-query equivalence is decidable (NP-complete, Chandra–Merlin 1977), and monotonicity is soundly-but-incompletely decidable over a whitelist (positive relational algebra
  - order-preserving scalar constructs).

This answers the doc's recurring open question — *how much can be decided statically vs. needs a declared guarantee?* — crisply: **the primitive must be a sufficient-condition analysis** (decide monotonicity soundly over a whitelist, require a declaration everywhere else, never push an unlicensed filter), exactly the §4.6 / §6.6 conservative contract.

7.4 The monotonicity primitive already exists — in three other shapes

smelt's Part 6 primitive is not speculative: three production systems implement a close analog, and their designs directly inform 6.2–6.5.

1. **ClickHouse `IFunctionBase::getMonotonicityForRange`** — the one production engine that reasons about function monotonicity *at plan time* to push a predicate on a derived expression onto a sorted source key. It returns a four-boolean verdict `Monotonicity { is_monotonic, is_positive (direction), is_always_monotonic, is_strict }` per function per range, consumed by `KeyCondition` to rewrite a predicate on `toStartOfDay(ts)`/`toDate`/`CAST` into a predicate on the primary key. **This is the closest structural analog to the verdict Part 6's classifier should return** — and the argument (§6.4, 6.7) for returning a verdict rather than a boolean. Altinity's write-up documents the *factor-transformation* trick for piecewise-monotone date functions (`toMonth`, `toDayOfWeek`) — monotone only within a range over which a coarser factor is constant — a technique smelt could reuse for `EXTRACT`-style expressions.
2. **Apache Iceberg partition transforms** — the cleanest *formalization*. Each transform carries a `preserves_order` boolean (`year`/`month`/`day`/`hour`, `truncate`, `identity` = true; `bucket`, `void` = false) plus a `project(predicate)` method: order-preserving transforms project a *range* predicate onto the partition field; `bucket` (a hash) can project *equality only*. **This is precisely smelt's licensing rule** — a range-predicate pushdown is sound iff the derivation is order-preserving — and smelt's static primitive is essentially a compile-time `preserves_order` classifier + `project`-style rewriter over `event_time = f(source_col)`.
3. **Delta Lake generated columns** — the inverse layout (partition column is the derived one), identical condition: a fixed whitelist of order-preserving generation expressions (`CAST(ts AS DATE)`, `YEAR`/`MONTH`/`DAY`/`HOUR`, prefix `DATE_FORMAT`, `SUBSTRING`) whose inverse image of a range is a range, letting a query on the source column produce a partition filter. A battle-tested enumeration smelt's whitelist (6.2) can mirror.

The negative baseline sharpens Part 4's "push at compile time, don't trust the engine" thesis (§4.4): Oracle, PostgreSQL, and SQL Server deliberately do *not* reason about monotonicity — *any* function wrapping the partition key defeats pruning, and the sanctioned workaround is to materialize the transform as a virtual/generated column. DuckDB (a smelt target) exploits a source-column range via zonemaps *once smelt has done the monotone rewrite*, but will not derive that rewrite from a derived-column predicate itself. So among common backends only ClickHouse would do this for you — and smelt is multi-backend, which is exactly why the rewrite must be smelt's job. All of them bottom out on a **hard-coded whitelist** because the general problem is undecidable (§7.3) — the same conservative posture 6.6 adopts.

7.5 Where smelt is novel

Across every window-forward system, "which column is the event-time clock" and "which input is the driving fact" are **declared by the user, never inferred**: Spark `withWatermark(col, delay)` + join direction; Flink `WATERMARK FOR ts` + `FOR SYSTEM_TIME AS OF`; Databricks `AUTO CDC ... SEQUENCE BY`; dbt `event_time=`; SQLMesh `time_column`; cube.dev `time_dimension`. All of them *trust* the declared column is monotone and that per-window evaluation equals whole-table evaluation; none proves it. The closest thing to an *inferred* version is Flink's internal `RelModifiedMonotonicity` (optimizer-derived per-column update-increasing/ decreasing metadata) — but even that is seeded from a user-declared watermark.

smelt's ambition to *derive* the event-time column from the SQL and *prove* it traces monotonically to a source partition column is stronger than anything shipped — best understood as "Flink's watermark attribute, but inferred-and- proven from the projection rather than annotated at the source." The theory (§7.3) bounds that ambition: the proof is necessarily incomplete (undecidable in general), so the honest design is *prove-where-you-can, declare-where-you-must* (§6.3) — reaching further toward inference than the annotate-only incumbents while keeping the declared escape hatch they rely on entirely.

Part 8 — Condition deep-dive: window functions, LAG/LEAD, and the two-layer lookback (B1 / C1)

Three Part-1 entries are really **one phenomenon** seen from three angles, and §4.3 row 3 already named it: a window whose frame reaches outside the run window forces *two* load-bearing filters — a **widened scan bound** at the source and an **exact output clamp** above the window operator. B1 (`incremental.rs:231`, the `PARTITION BY ⊇ partition_column` gate, overridable via `allow_window_functions`), C1 (`incremental.rs:574` / `safety.rs:100`, bare LAG/LEAD → `NotDerivable`, detected at `source_bounds.rs:240`), and the `UNBOUNDED PRECEDING` → per-partition fallback (`incremental.rs:72`, a *non-rejection*) are the three faces of the same frame-reach question. This part works the cluster as a unit, in the four-step frame of Parts 2/3/5, and shows the whole cluster reduces — like §2.5, §4.6, and §5.4 — to the Part 6 monotonicity primitive plus one new quantity it must return: the **finite lookback margin** the frame reaches back.

8.1 Why it is rejected (three faces, one cause)

`inject_time_filter` (`transformer.rs:272`) writes the window predicate on the **output**; `inject_source_filters` (`transformer.rs:65`) prunes each **source scan** on the *same* run window. For every construct worked so far the two windows **coincide** (§4.3): a transparent body, a distributing `UNION` branch, a fact-side join filter — all need to scan exactly the rows they emit. A window function is the first construct where they legitimately **diverge**: the output row for event-time `t` is computed from *other* rows in its frame, so producing window `[lo, hi)` requires reading rows *below* `lo`.

The three current behaviours are three conservative responses to that divergence:

- **B1** sidesteps it by demanding `PARTITION BY ⊇ partition_column`. When the window partitions *by* the partition column, every frame stays inside a single output partition, the two windows coincide again, and no lookback is needed — so B1's slice is the **zero-lookback** slice and needs neither the monotonicity primitive nor an `ORDER BY` constraint. Anything else is gated behind `allow_window_functions`, i.e. refused by default.
- **C1** refuses bare LAG/LEAD because their reach is a **row** offset, and a row offset has no finite *time* bound to widen the scan by (`NotDerivable`, `source_bounds.rs:240`).
- **`UNBOUNDED PRECEDING`** is not rejected but degraded to `BatchSafety::PerPartitionOnly` (`incremental.rs:72`): its reach is the whole partition, so no *finite* widening recovers it — the only sound refresh is to recompute each touched partition entire.

None of the three is a statement that windowing is incompatible with incremental refresh. Each is a conservative stand-in for the missing quantity: **how far back in event-time does the frame reach, and is that distance finite and derivable?**

8.2 The correctness result: a bounded frame is a widened-scan commutation

Model the window operator as ω. Producing output window `W = [lo, hi)` needs

```
π_W( ω(R) )  =  π_W( ω( σ_[lo−k, hi)(R) ) )
```

to hold, where k is the frame's backward reach in event-time. Read literally: compute ω over a scan **widened backward by k**, then clamp the output to `[lo, hi)`. This is exactly σ/ω commutation *with a margin* — the general form of the Part 4 pushdown walk when the operator is not filter-transparent. It holds under two conditions, both supplied by the Part 6 primitive plus the frame:

1. **`ORDER BY` is the monotone event-time.** ω's frame is defined over the `ORDER BY` key; for a `RANGE` frame to correspond to an event-time interval, that key must be the model's `event_time` (or a monotone image of it, §6.2). Then "the frame reads back k" is a statement about event-time, and the scan bound `lo − k` on the source **partition column** selects exactly the rows the frame reads (the same interval-preimage-is-an-interval property of §6.1).

2. **`PARTITION BY` ⊇ `partition_column` is *not* required** — this is the relaxation. If the window partitions by, say, `user_id` (crossing day partitions), the frame for `(user u, day D)` reaches into partitions `D−k … D`; the widened scan `[lo−k, hi)` covers them, and the output clamp discards the widened rows so no partition outside `W` is rewritten. Both layers are load-bearing: drop the widening and the early-window rows are understated (proven below, **W2**); drop the clamp and rows below `lo` are written twice.

So the two windows differ by **exactly the lookback k**, and k is a *static* property of the frame — not a data-dependent guess. That is the whole novelty of this cluster: smelt can **derive** the margin from the SQL where streaming engines make the user declare it (§8.6).

This two-layer design is a *change* from the shipping runtime, not a description of it. Today the runtime widens **both** the outer clamp *and* the DELETE to `[run_start − k, run_end)` (§4.2) — it *re-writes* the margin rows rather than merely reading them, and because the scan is only widened by k, the re-written margin is recomputed from clipped frames (the confirmed §4.2 under-read; a widened-write design would need a 2k scan). Under the exact clamp proposed here, k suffices because the margin is read, never written — and writes become strictly partition-disjoint, which is what Part 11's parallelism claim rests on.

8.3 Frame taxonomy: only `RANGE`-with-`INTERVAL` yields a derivable time bound

The dividing line is **what the frame counts**. SQL has three frame modes, and only one measures the `ORDER BY` key in its own units:

Frame	Counts	Time reach k	Verdict
`RANGE BETWEEN INTERVAL 'k' PRECEDING AND CURRENT ROW`, `ORDER BY` = event-time	value distance on the key	**exactly k** — the interval *is* the margin	**derivable → two-layer safe**
`RANGE … UNBOUNDED PRECEDING`	whole partition down to −∞	∞	no finite scan; **per-partition recompute** (`BatchSafety::PerPartitionOnly`
`ROWS BETWEEN n PRECEDING …`	*rows*, regardless of their event-time spacing	unbounded in time — n rows can span an arbitrary interval on a sparse stream	**`NotDerivable` → reject** (unless density declared)
`GROUPS BETWEEN n PRECEDING …`	distinct peer values of the `ORDER BY` key	unbounded in time for the same reason	**`NotDerivable` → reject**
bare `LAG(col, n)` / `LEAD(col, n)`	one row n positions away — a degenerate `ROWS n PRECEDING/FOLLOWING`	unbounded in time	**`NotDerivable` → reject** (this *is* C1)
default frame (`ORDER BY` with no explicit frame)	`RANGE UNBOUNDED PRECEDING AND CURRENT ROW`	∞	per-partition recompute

The crucial pairing: `RANGE INTERVAL` measures the frame in the same clock the partition column is derived from, so k folds directly into the source bound — this is the same offset-folding `source_bounds` already does for `col + INTERVAL` (Form B, `source_bounds.rs:359`; `MONTH ≈ 30 days`, `:506`). `ROWS`/`GROUPS` count *positions*, which map to a time distance only through the data's density — unknowable statically, hence `NotDerivable`. `LAG`/`LEAD` are simply the single-row case of `ROWS`, which is why C1 already lands there. And a `LAG`/`LEAD` carrying **no** n beyond 1 is still a *row*

offset, not a time offset: the C1 rejection is correct, not merely conservative — a user with one event in the window has a predecessor arbitrarily far back (proven below, **W4**).

Forward (`FOLLOWING`) reach is the unworked mirror of this table. Every row above treats backward reach only. A bounded `RANGE … INTERVAL 'a' FOLLOWING` frame (or `LEAD`, its row-offset cousin, which C1 already rejects for the same reason as `LAG`) reads rows *after* the current one, which changes the problem twice over: the scan must widen **forward** by `a` (the `after_secs` half of the source filter, `transformer.rs:83–84`, exists for exactly this — though `source_bounds` Form A currently parses `PRECEDING` frames only), and — sharper — window `W`'s output is not *settled* until the source is complete through `hi + a`: a window computed as soon as it closes will silently differ from a later full refresh once the following rows arrive. The two sound treatments are watermark-style delay (do not run `W` until `now ≥ hi + a`) or tail-rewrite (each run re-writes `[run_start − a, run_end)`, whose scan must then widen backward by a *plus* any preceding reach — the §4.2 composition trap again). Neither is analysed in this document yet; recorded as an open question.

`UNBOUNDED PRECEDING` deserves the sharper statement §4.3 row 3 implied: it is per-partition-recomputable **only when B1 holds** (the partition is self-contained). If the window *both* is unbounded *and* crosses the partition column, even per-partition recompute is unsound — the running total for one day depends on prior days — and the honest verdict is full recompute. The current `PerPartitionOnly` fallback is therefore correct precisely in the B1 regime and must not be extended to the cross-partition case without widening to full.

The cross-partition running total is not this cluster's problem to solve — it is `refresh: cumulative`. An `UNBOUNDED PRECEDING` sum whose reach crosses the partition column is exactly a *cumulative aggregate*: a stored state that grows forward across every window. `smelt` already models this on a **separate refresh axis** (the D3 rejection of incremental-`refresh: cumulative` in Part 1), specified in `docs/specs/cumulative_aggregate.md` and executed via the `merge_into` backend primitive rather than partition-scoped DELETE+INSERT. So the window-cluster verdict for a genuinely cumulative reach is *not* "reject" but "**this is a cumulative model, not an incremental one** — use the cumulative refresh," and the two workstreams should agree on where that line falls (Open Questions).

8.4 The sharpened eligibility condition

A window model is incrementalisable with a **derived** lookback exactly when:

1. **The `ORDER BY` key is a monotone image of the source event-time** — the Part 6 primitive returns `Traceable{ source, source_column, offset }` for it. This subsumes B1's implicit assumption and replaces the substring-free B1 check with the same trace the other three consumers call (§6.4).
2. **The frame is a bounded `RANGE` with a temporal `INTERVAL`** — giving a finite k. `ROWS`/`GROUPS`/`LAG`/`LEAD`/`UNBOUNDED` fail this and stay on their current paths (reject or per-partition).
3. **The margin composes with the source bound**: scan window = `[run_start − k − offset, run_end)`; output clamp = `[run_start, run_end)`. k is the frame interval; `offset` is any monotone shift the primitive folded out (§6.2). Where k is a non-uniform interval (`MONTH`/`YEAR`), it rides as the `Symbolic` offset of §6.2 / the open question in §6.7.

Under (1)–(3) the current B1 gate is **too strict** in one direction (it rejects the safe cross-partition bounded-`RANGE` window) and the C1/`UNBOUNDED` paths are **correct** (their reach is genuinely underivable / infinite). The relaxation is therefore narrow and precise: admit `PARTITION BY` ⊉ `partition_column` iff the `ORDER BY` is monotone-event-time *and* the frame is bounded `RANGE`, deriving k as the second layer's widening.

8.5 Empirical confirmation (DuckDB v1.4.4)

Harness: `docs/research/harness/20260701-window_incremental.sql` (run with `duckdb -box < …`). As in §2.3/§3.5/§5.5, each property reports violating rows via `(L EXCEPT ALL R) ⊎ (R`

EXCEPT ALL L)|; 0 = the identity holds. The source is a dense per-user daily grid; the model is a per-user trailing sum (RANGE INTERVAL 2 DAY PRECEDING, PARTITION BY user_id — deliberately *not* the partition column, the case B1 rejects today).

Property	Violations	Meaning
W1 bounded RANGE frame, scan widened by the 2-day margin	**0**	two-layer identity holds: widened scan + exact clamp = full refresh (§8.2)
W2 same frame, scan **not** widened (clamped to the output window)	**80**	dropping the widening understates the frame at the window's early days (hazard reproduced)
W3 UNBOUNDED PRECEDING running total, only a finite margin applied	**120**	no finite widening recovers a −∞ reach; must be per-partition/full recompute (§8.3)
W4 bare LAG, fixed 2-day scan margin on a sparsified stream	**40**	a *row* offset has no finite *time* bound; the true predecessor sits outside any fixed margin (C1 confirmed)

W1=0 is the positive result — the derived lookback makes a cross-partition window incremental≡full. W2/W3/W4 are the three hazards the taxonomy predicts, each non-zero for exactly the reason §8.3 gives: an insufficient margin (W2), an infinite reach (W3), and a time-unbounded row offset (W4).

8.6 Prior art — everyone declares the lookback; smelt can derive it

Window-forward engines universally treat the scan margin as a **declared** number, because they cannot see the frame at plan time the way smelt can:

- **Flink** requires an OVER window to be ORDER BY the **rowtime attribute** (its monotone event-time) — precisely §8.4 condition (1) — and its OVER output is *append-only* once the watermark passes, versus Top-N which is *updating*. The watermark's allowed-lateness is the declared margin; state is retained exactly that far back. Flink *validates* the ORDER BY-by-time requirement but never derives the reach.
- **Spark Structured Streaming** pairs window() with withWatermark(col, delay): delay is the declared lateness, and windows whose end falls before the watermark are finalized and their state evicted. The margin is a user knob, not a frame derivation.
- **Databricks Enzyme** ships WINDOW_WITHOUT_PARTITION_BY — analytic functions are incrementalizable **only with** a PARTITION BY. That is exactly smelt's current B1 regime (zero-lookback intra-partition), externally confirming B1 as a *correct* slice while saying nothing about the cross-partition bounded-RANGE case smelt can additionally prove.
- **Snowflake Dynamic Tables** classify window functions as *blocking operators*: supported, but they push the model toward full refresh, and *non-identical PARTITION BY across window functions* blocks incremental — a coarser, cost-driven version of the same partition-alignment reasoning.
- **BigQuery MVs** exclude **all** analytic functions — the maximally conservative baseline (BigQuery does no frame analysis at all).
- **dbt microbatch** exposes lookback = N (reprocess the last *N* batches) and **SQLMesh** tracks intervals in state; both address **late-arriving source data**, a *different* axis from the frame reach (see below), and both are declared, never derived.
- **DBSP** (§7.3) makes the algebra explicit: a frame aggregation is a **non-linear** operator requiring nested integration/differentiation over the frame; a **bounded** frame is bounded state

(cheaply incremental — the W1 slice), an **unbounded** frame integrates the whole partition (the W3 full-integration case). smelt's bounded-RANGE-vs-UNBOUNDED split *is* DBSP's bounded- vs- unbounded-state split. **Dataflow watermarks** (§7.3) are the reason a lookback exists at all: a monotone completeness bound past which late data is dropped.

Own analysis — the lookback decomposes into two independent margins, and smelt uniquely derives one of them. The scan window must be widened backward for two *orthogonal* reasons:

- **(a) computation reach** — how far the model's own window frame reads (`RANGE INTERVAL` k → k). This is a **deterministic function of the SQL** and is exactly what smelt can derive.
- **(b) source lateness** — how late a source row may arrive relative to its event-time. This is a **data/pipeline property**, invisible in the SQL, and is what dbt `lookback` and Spark `withWatermark` declare.

Streaming engines fuse (a) and (b) into a single declared allowed-lateness because they cannot separate them. smelt can compute **(a) exactly** from the frame and needs a declaration **only for (b)** — and the two simply **add**: total scan margin = k (derived) + declared source-lateness (default 0). This is the same "prove-where-you-can, declare-where-you-must" posture as §7.5 / §6.3, now made quantitative: smelt derives the computation-reach term and leaves only the genuine data-property term to declaration. No surveyed engine makes this split; it falls straight out of smelt's compiler-not-engine identity (§4.4).

8.7 Recommendation for the window cluster

Ship the **bounded-`RANGE` two-layer slice**, and fold B1/C1/the `UNBOUNDED` fallback into one frame-reach classifier that reuses the Part 6 primitive:

1. **Replace B1's `PARTITION BY ⊇ partition_column` gate with a frame-reach analysis.** Call `trace_event_time` on the window's `ORDER BY` key; require `Traceable`. If the frame is a bounded `RANGE INTERVAL`, admit the window with a derived lookback k even when `PARTITION BY` crosses the partition column — the case B1 rejects today. The current zero-lookback intra-partition slice remains a special case (`k = 0`).
2. **Emit the two layers explicitly.** The classifier returns, alongside the Part 6 trace, the **frame margin k**; `inject_source_filters` widens the scan to `[run_start − k − offset, run_end)` while `inject_time_filter` keeps the exact `[run_start, run_end)` output clamp. Where a declared source-lateness exists (§8.6), add it to k.
3. **Keep rejecting / degrading the un-derivable reaches, by name.** `ROWS`/`GROUPS` frames, bare `LAG`/`LEAD` (C1), and any window whose `ORDER BY` is *not* monotone-event-time stay `NotDerivable` — but with a message that names the frame mode and says *why* a row/peer offset has no time bound, not a blanket "window functions not supported." `UNBOUNDED PRECEDING` / default frame stays `PerPartitionOnly` **when B1 holds** and widens to full recompute when the window crosses the partition column (§8.3).

This slots into Part 4 as the case that finally *needs* two layers: for the transparent slice the output-clamp and scan windows collapse into one filter (§4.3), but a bounded frame is the construct where they legitimately differ by exactly the derived k — the irreducible two-layer result §4.3 row 3 predicted, now measured (W1) and bounded (W2–W4). Like joins (§5.7), the window cluster is not a new mechanism but the frame-shaped instance of the same commutation walk, blocked on the same monotonicity primitive plus one extra returned scalar: the lookback margin.

Part 9 — Shorter conditions

Four of the Part 1 rejections do not warrant a full Part-2-style deep-dive: their correctness question is settled by an argument already made elsewhere in this document, and the work is to *apply* that argument, not discover it. This part disposes of them together. Each still runs the standard frame

— *why rejected → correctness law or mechanical limit → safe relaxation → recommendation* — but leans on Parts 2–8 rather than re-deriving. The window-function / LAG/LEAD / two-layer-lookback cluster is the one remaining condition that *does* need its own deep-dive (Part 8); it is deliberately excluded here.

9.1 — B5 non-determinism: split the bucket

Why it is rejected. `incremental::detect` (`incremental.rs:288`) rejects any model whose SQL calls a non-deterministic function — `RANDOM`, `NOW`, `UUID`, `CURRENT_DATE`, `CURRENT_TIMESTAMP`, … — as a single class, overridable via `allow_nondeterministic`. The stated fear is the incremental ≡ full invariant: a function that returns different values on re-evaluation makes a windowed rebuild disagree with a full refresh.

Correctness law or mechanical limit. The single-class treatment is a mechanical over-approximation. The functions split on *when* the value is resolved, and only one half actually breaks the invariant:

- **Row-nondeterministic** — `RANDOM()`, `UUID()`, `GEN_RANDOM_UUID()`. A fresh value *per row, per evaluation*. Re-running any window re-rolls the value, so the stored partition after an incremental run differs from a full refresh of the same range. These genuinely violate incremental ≡ full and must stay rejected. (They are also the reason `unique_key` MERGE on a random column is meaningless.)
- **Run-deterministic** — `NOW()`, `CURRENT_DATE`, `CURRENT_TIMESTAMP`, `CURRENT_USER`. Resolved *once per statement execution* and identical for every row in that run. These do not vary within a run; they vary *between* runs. Whether that breaks the invariant depends entirely on whether the incremental sequence and the reference full refresh are pinned to the *same* value.

This is the same run-vs-row distinction §6.6 already draws for the monotonicity primitive, where a run-deterministic clock is called out as "admissible as an outer clamp, never as a pushed source filter." B5 is the projection-side twin of that rule: a run-deterministic function is safe to *emit* (it lands one constant in every row of the run), but it is never a *source-traceable* event-time and so can never license a pushed source filter — it is `NotTraceable` in the Part 6 classifier, admissible only above the scan.

Safe relaxation. Admit the run-deterministic functions by **pinning** them to a single compile-time constant, resolved once and substituted into the emitted SQL for every window of the run. The subtlety that makes this correct — and the trap if ignored — is that the pinned value must be the value the *reference full refresh would use*, not "the wall clock at the moment each window compiles." Concretely:

- A full refresh over `[t0, tN)` evaluates `NOW()` once, to some `T_full`.
- An incremental sequence of adjacent windows `[t0,t1), [t1,t2), …` must therefore substitute *that same* `T_full` into every window, not a per-window now. If each window pinned its own compile-time clock, `DATEDIFF(NOW(), event_ts)`-style expressions would drift window-to-window and diverge from the full refresh.

So the guarantee is not "`NOW()` is run-deterministic" alone — it is "`NOW()` is pinned to a single value shared across the whole incremental sequence *and* the full-refresh oracle." Given smelt compiles the plan, it already controls this substitution point; the value belongs with the run window the runtime already threads (`execute.rs`). Row-nondeterministic functions cannot be pinned (the value is intrinsically per-row) and stay rejected.

Invocation scope — the honest limit of pinning. The pin is coherent *per invocation*. A backfill that processes many windows in one invocation shares one `T_full`, and incremental ≡ full holds against a full refresh evaluated at that same instant. An *ongoing schedule* is different: each day's run is its own invocation with its own pin, so the stored table becomes a patchwork of clocks that equals **no** single-instant full refresh — and no pinning scheme can fix that short of freezing time at the first run forever. That patchwork is usually exactly what users want from `NOW()` in a projection

(loaded_at-style audit columns record the run that produced the row), but it must be named as a **contract change, not a preserved invariant**: pinning restores incremental ≡ full *within one invocation*; across invocations, any column derived from the pinned clock is a documented divergence (or is excluded from the invariant outright).

This mirrors the industry line in §7.2: Snowflake rejects non-deterministic functions *in the SELECT projection* but permits CURRENT_* in a WHERE (where it prunes rather than materialises); Databricks Enzyme's EXPRESSION_NOT_DETERMINISTIC is the same blanket B5. smelt's refinement — pin-and-admit the run-deterministic subset — is slightly ahead of both, and cheap because smelt owns compile-time substitution.

Recommendation. Split B5 into two guards. Keep a hard rejection for row-nondeterministic functions (RANDOM/UUID/…). Admit run-deterministic functions (NOW/CURRENT_DATE/CURRENT_TIMESTAMP/…) by default, implemented as compile-time pinning to a single run-shared constant, with the invariant test being: *the same pin feeds every incremental window and the full-refresh oracle, within one invocation* (across scheduled invocations the pinned-clock columns are a documented divergence — see the invocation-scope caveat above). Retain allow_nondeterministic only as the escape hatch for the row-nondeterministic class. Emit an honest message naming *which* function is the problem, not "non-deterministic function" as a category.

9.2 — Non-additive (holistic) aggregates: a delta-engine rejection that does not transplant

Why it looked like a gap — and why it is not one. §7.2 obs. 2 surfaced this from the industry comparison: Snowflake Dynamic Tables and BigQuery MVs both explicitly exclude MEDIAN, PERCENTILE_CONT/PERCENTILE_DISC, and exact COUNT(DISTINCT) from incremental refresh, and smelt's catalogue has no condition naming non-additive aggregates as a class. The first draft of this section proposed a new **B7** gate mirroring those whitelists. **On closer inspection the transplant is wrong: in smelt's refresh regime these aggregates are safe.** (Corrected 2026-07-02.)

Why the industry rejection does not apply here. Snowflake, BigQuery and Enzyme are *delta* engines (§7.1): they maintain a view by **merging partial aggregates** — this refresh's partial state combined with previously-stored state. Decomposability (a bounded partial plus an associative merge) is precisely the property *merging* needs, and holistic aggregates lack it. smelt's window-forward DELETE+INSERT never merges partials: A4 (§9.4) requires the GROUP BY key ⊇ partition_column, so **every group lives entirely inside one window and is recomputed from scratch, in full, by the one run that writes its partition**. There is no cross-window combination step for decomposability to matter to. A per-day MEDIAN(latency) with GROUP BY day is computed over exactly the rows a full refresh would give that group; the values are identical — including under late-data reprocessing (a re-run rewrites the whole partition from the whole group) and the Part 10 open-partition rule (the open partition is recomputed entire each run). The only way to break it is the g_run < g_part misconfiguration of §10.2 — which breaks SUM identically, so it is Part 10's gate, not an aggregate-class gate.

Empirically (DuckDB v1.4.4, harness docs/research/harness/20260702-holistic_aggregate.sql): a partition-aligned MEDIAN over two adjacent windows vs. a full refresh — **0** violating rows, under the same |(L EXCEPT ALL R) ⊎ (R EXCEPT ALL L)| test as every other harness property.

Where decomposability *does* bite (the analysis is not wasted). The classification becomes load-bearing exactly where a **partial-merge regime** exists:

- **refresh: cumulative** (D3, cumulative_aggregate.md) maintains running state via merge_into. A running SUM/COUNT/MIN/MAX is a bounded partial; a running MEDIAN is not. The decomposability whitelist (SUM/COUNT/MIN/MAX/AVG-as-SUM÷COUNT decomposable; MEDIAN/PERCENTILE_*/MODE/exact COUNT(DISTINCT) holistic; unrecognised heads fail closed to holistic, §6.6) belongs in the **cumulative** spec's eligibility rules, not in the incremental catalogue.

- **Cross-window groups**, if group alignment (A4) is ever relaxed — a group spanning windows would need partial-merge to avoid re-reading other windows.
- **The `MIN`/`MAX` append-only corollary lives there too.** Within the aligned full-rewrite regime `MIN`/`MAX` need no append-only caveat (the whole group is recomputed every time). It is *merging* an extremum forward that relies on no-deletes: a delta engine (Flink, §7.2 obs. 2) must keep retraction state to recompute a `MIN` whose holder is deleted; a cumulative smelt model would inherit the same caveat.

Recommendation. No B7 gate for the incremental regime — a partition-aligned holistic aggregate is safe and must not be rejected. Instead: (a) carry the decomposability classification into the cumulative-aggregate spec, where merging makes it a genuine eligibility law; and (b) treat this as a methodological caution for §7.2: the industry comparison validates the catalogue only where the *refresh mechanism* behind each published rule matches smelt's — copying a delta-engine whitelist entry into a whole-partition-rebuild regime would have produced a spurious rejection.

9.3 — B2 `HAVING` / B6 `DISTINCT` / B3 `LIMIT`

Why they are rejected. All three are override-gated Pathway-A B-group checks (`incremental.rs:248`, `:302`, `:261`), each carrying an `allow_*` escape hatch. The question this part asks is narrower than the others: can any of the three move from *override-gated* to *safe-by-default*?

Correctness law or mechanical limit — resolved by §3.2's commutation test. The governing fact is whether the construct commutes with the injected window predicate `σ_event_time` (§3.2, §4.1). Run each through it:

- **`LIMIT` (B3) — never commutes. Keep gated.** `LIMIT` selects *k* rows from an ordered (or arbitrary) set; the *k* rows chosen from a single window are not the *k* rows chosen from the full range. `σ_event_time(LIMIT_k(R)) ≠ LIMIT_k(σ_event_time(R))`. This is precisely the Q5a hazard the harness reproduces (§3.5, 30 violating rows). `ORDER BY … LIMIT` (top-N) is the same wall. No safe slice; the override is the only correct path, and even then the result is not incremental ≡ full — it is "the user asserts they don't care."
- **`DISTINCT` (B6) — cross-window dedup does not commute. Keep gated (with one narrow exception).** `SELECT DISTINCT` over columns spanning multiple windows can collapse rows that a per-window rebuild would keep separate, or vice versa. This is the non-monotone `DISTINCT`/`GROUP BY` boundary of §7.3 and the industry line of §7.2 (Snowflake/Enzyme both reject plain `DISTINCT`). The one case that *is* safe — `DISTINCT` where the dedup key ⊇ `partition_column`, so duplicates can only ever fall in the same window — is the exact `DISTINCT`-as-degenerate-`GROUP BY` mirror of §3.2 row 3, and if pursued should be handled by the same group-aligned machinery as A4/§9.4, not by relaxing B6 wholesale. Absent that, keep gated.
- **`HAVING` (B2) — has a genuine group-aligned safe slice.** `HAVING` is a filter on aggregated groups. When the `GROUP BY` key ⊇ `partition_column` (the §3.2-row-3 condition again), every group is window-local, so the `HAVING` predicate is evaluated over exactly the rows a full refresh would give that group — it commutes, because the filter never spans a window boundary. `SELECT date, SUM(x) FROM … GROUP BY date HAVING SUM(x) > 100` produces the same surviving groups incrementally as in full, since each date's `SUM(x)` is fully materialised within its window. This is a real safe-by-default slice, gated today only because B2 is coarse.

Recommendation. Per construct:

- **B3 `LIMIT`** — keep as hard override-gated; there is no safe-by-default slice, and even the override is a "user accepts divergence" signal, not a correctness claim.
- **B6 `DISTINCT`** — keep gated by default; treat the `DISTINCT`-key ⊇ `partition_column` case as part of the group-aligned aggregation work (§9.4 / A4), not a standalone relaxation.
- **B2 `HAVING`** — relax to safe-by-default **when** `GROUP BY` key ⊇ `partition_column` (window-local groups), rejecting otherwise. This rides on the same partition-alignment check A4 already performs (§9.4) — no new analysis, just conditioning B2 on the alignment A4 establishes.

All three collapse onto one prerequisite: the partition-alignment check. B2's safe slice and B6's

narrow exception are both "the GROUP BY/dedup key contains partition_column," which is §9.4's business.

9.4 — A4 partition_column in GROUP BY

Why it is rejected. A4 (incremental.rs:181) requires, for aggregate models, that partition_column appear in the GROUP BY. This is a genuine correctness requirement, not a conservative one: if the partition column is not a grouping key, a single output group spans multiple partitions, and a partition-scoped DELETE+INSERT cannot rewrite that group correctly. A4 is the check that makes "each group lives in exactly one window" (the §3.2-row-3 / §9.2 / §9.3 safe-slice precondition) *true*. It should stay.

The real work — where A4 must run. A4 today runs analyze_select over the **outer, flat** query (incremental.rs:132+). Parts 2 and 3 both flagged that this location becomes wrong once aggregation can appear *inside* a construct:

- **Inside a UNION branch (§2.6).** §2.6 states A3–A6 "must run **per branch**." For A4 specifically: each aggregating branch has its *own* GROUP BY, and each must independently include that branch's partition_column projection. A branch that aggregates without partition alignment is unsafe even if its siblings are fine — A4 must be evaluated once per branch, against that branch's SELECT/GROUP BY, not once over the set-op as a whole (which does not even parse as a single GROUP BY).
- **Inside a subquery / CTE body (§3.6).** §3.6 states A3–A6 "must resolve against the **subquery's** SELECT list, not the outer one that just says SELECT *." For A4: when the aggregation lives in the derived-table/CTE body (the group-aligned case of §3.2 row 3), the GROUP BY to check is the *body's*, and partition_column must be a body grouping key traced through to the outer projection.

So A4 does not change as a *rule* — the correctness statement is unchanged — but its *evaluation site* must follow aggregation wherever the per-branch (§2.6) and per-body (§3.6) refactors relocate it. This is the same "lift the flat-model checks to the construct's actual scope" theme both those sections raise; A4 is simply the member of A3–A6 whose relocation *also* unlocks safe slices elsewhere (B2's HAVING, §9.3; MIN/MAX group-local aggregation, §9.2; the DISTINCT-key exception, §9.3). It is therefore the load-bearing one to get right.

Recommendation. Keep A4 as a correctness law. As part of the §2.6 (per-branch) and §3.6 (per-body) refactors, make A4 a **scoped** check: it takes a SELECT context (a branch, a subquery body, or the flat outer query) and verifies partition alignment *within that scope*. Expose its verdict ("this scope's groups are partition-local") as a reusable signal, since §9.3's HAVING safe slice and DISTINCT exception condition on exactly it — and §9.2's withdrawal of B7 rests on it too (partition-local groups are *why* holistic aggregates are safe). One partition-alignment predicate, evaluated at the right scope, several dependents.

Part 10 — Output-time granularity and the run-window / partition alignment

Every condition worked so far asks *which column is the clock* and *whether a window filter commutes*. This part isolates a constraint that is **orthogonal to all of them** and surfaced by one concrete use case — **aggregating a daily input into a monthly output**. It is not an eligibility gate on a SQL construct; it is a relationship between three granularities that must line up for partition-scoped DELETE+INSERT to be sound, whatever the SQL looks like.

10.1 The three granularities

An incremental model has three time granularities that need not coincide:

1. **The source clock** `g_src` — how finely source rows are stamped (e.g. per- second `created_at`).
2. **The output partition** `g_part` — the granularity of `partition_column`, the unit DELETE+INSERT rewrites atomically (e.g. `DATE_TRUNC('month', …)` → month).
3. **The run window** `g_run` — the span a single incremental run processes (the `[run_start, run_end)` the runtime threads).

Part 6's monotonicity primitive handles the *relationship between `g_src` and the event-time transform*: `DATE_TRUNC('month', created_at)` is a monotone image of the source clock, so "emit a monthly `event_time` from a per-second source" is `Traceable` and needs no new machinery (§6.2). That is the **transform** axis, and it is fully covered. What Part 6 says nothing about is the relationship between `g_part` and `g_run` — the **cadence** axis — because it is not a property of any expression.

10.2 The constraint: a run must rewrite whole partitions

Partition-scoped DELETE+INSERT deletes an entire partition and reinserts the rows the run computed for it. That is sound **iff every partition the run touches is recomputed in full by that run** — i.e.

> **`g_run` must be a whole multiple of `g_part` (a run covers whole partitions), or the touched partitions must be recomputed entire (per-partition recompute).**

The daily→monthly case makes the failure vivid. Partition by month, but run a *daily* window `[D, D+1)`. The run produces one day of the month's aggregate, then DELETE+INSERT **deletes the whole month's partition and reinserts a single day** — silently discarding the other 29 days. The invariant breaks not because `DATE_TRUNC('month', …)` is non-monotone (it is perfectly monotone) but because the *run cadence is finer than the partition it rewrites*. The dual also fails harmlessly-but-wastefully: partition by day, run monthly — each run rewrites 30 day-partitions, which is *correct* (whole partitions) but simply a coarser cadence.

So the rule is directional — but coarseness alone is not enough: **each run's window must cover whole partitions**, which requires *both* `g_run` a whole multiple of `g_part` *and* the run boundaries aligned to partition boundaries. A month-long run window starting mid-month is `g_run = g_part` yet spans two partial months and fails the same way as the daily run. (And when a derived lookback widens the *write* window backward, §4.2, the alignment requirement applies to the widened window's lower edge too.) A monthly partition therefore demands month-aligned, monthly-or-coarser run windows, or the incomplete month must be handled by recompute.

10.3 The incomplete-final-partition corollary

Even with `g_run` ≥ `g_part`, the *current* partition is a moving target: mid-month, the month's aggregate is incomplete and will change as more days arrive. Under DELETE+INSERT this is fine **provided each run recomputes the whole month-to-date**, which `g_run` ≥ `g_part` already guarantees (a monthly run reprocesses the whole open month every time). The hazard is only the finer-cadence mistake of §10.2: a daily run that touches the open month writes a partial value and never revisits the earlier days. This is the same "settled vs. open window" reasoning as watermarks (§7.3) — the open partition is never settled until `g_part` fully elapses — applied to the *partition* granularity rather than the event-time completeness bound.

10.4 Relationship to A4 — the two alignment laws are duals

§9.4's A4 (`partition_column` ∈ `GROUP BY`) and this constraint are **dual halves of one alignment story**:

- **A4 aligns groups → partitions.** Each output *group* must live in exactly one partition (so a partition can be rewritten without touching other groups).

- **§10.2 aligns runs → partitions.** Each *run* must rewrite whole partitions (so a partition is never left half-computed).

A4 is checked today; the run↔partition constraint is only **half-built**. A boundary-alignment validator exists — `validate_run_window_alignment` (`smelt-runtime/src/windowing.rs:205`) checks that a run window's boundaries land on the declared `timeseries.granularity` grid (Monday for weeks, the 1st for months, …) — but it is not called from the live incremental path (`compute_incremental_windows`, `windowing.rs:63`, never invokes it), and smelt has a *single* declared granularity, so nothing cross-checks the partition-column *transform's* unit (`DATE_TRUNC('month', …)`) against that declaration or the run cadence. A user can declare `granularity: day`, partition by `DATE_TRUNC('month', …)`, and run daily — the mismatch is invisible. Both alignment laws are preconditions for partition-scoped DELETE+INSERT to equal a full refresh; A4 covers the *group* side, §10.2 the *cadence* side. A complete eligibility model owes a check for the second, most naturally as a validation that the configured/derived run granularity is ≥ the `partition_column` granularity (both of which smelt can read: the partition granularity from the `DATE_TRUNC`/`CAST` unit the monotonicity primitive already parses, §6.2, and the run granularity from the run window the runtime threads).

10.5 Recommendation

- **Treat granularity as a validation, not an eligibility gate.** The SQL is eligible; what needs checking is a *configuration* invariant: `g_run ≥ g_part`. Derive `g_part` from the partition-column transform unit (`DATE_TRUNC('month', …)` → month) via the Part 6 primitive, compare against the run cadence, and reject (or auto-coarsen the run window to `g_part`) when the run is finer — and wire the dormant `validate_run_window_alignment` boundary check (§10.4) into the live run path while at it.
- **Handle the open partition by recompute-of-touched-partition**, the same `PerPartitionOnly` mechanism §8.3 already uses for `UNBOUNDED` frames — the open month is recomputed entire on each run until it closes.
- **Keep this orthogonal to Part 6.** The transform (monotone image, any granularity) is Part 6's job; the cadence relationship is a separate, cheaper check that does not touch the monotonicity classifier.

Part 11 — Window independence: run order and parallelism

Every part so far asks whether a model *can* be incremental. This part asks a different, **execution-time** question the governing invariant is silent on: given that a model is incremental, **must its windows be run in order, or may they run out of order / in parallel?** The "incremental ≡ full over adjacent windows" contract fixes the *result*, not the *schedule* — two orchestrators that run the same windows in different orders should both reproduce the full refresh, but only if the model permits it. The user pattern that motivates this: a model that references *previous partitions* cannot have its partitions computed concurrently or out of order.

11.1 The discriminator: does window `W` read the source, or its own output?

There is one clean test. Producing output window `W` reads some set of rows; the only question is *where those rows come from*:

- **Window-independent** — `W` reads only the **source** (its own window, plus a bounded lookback into *earlier source rows*, Part 8). No output window depends on the *computed result* of any other. Windows are mutually independent and may be scheduled in **any order, concurrently**.
- **Sequentially-dependent** — `W` reads the model's **own prior output** or an **accumulated cross-window state**. Then `W_n` needs `W_{n-1}` to have been computed first; the windows form a chain and must run **strictly in order, no parallelism**.

Everything turns on source-vs-self, and it decides parallelism, not eligibility: a sequentially-dependent model can still be perfectly incremental — it just cannot be run out of order.

11.2 The whole safe slice of this document is window-independent

A key structural fact ties this back to Part 4 — with one load-bearing premise: **lookback must widen the source *scan*, never the output *write*.** Under the Part 8 exact-clamp design, the output clamp restricts what a run *writes* to `[run_start, run_end)`, while any lookback margin k (Part 8 frames, the §5.3 interval-join band) only widens what it *reads* from the source. Then:

- **Writes are always partition-disjoint.** No run ever writes outside its own window, so two concurrent runs touch disjoint partitions — the DELETE+INSERT / `unique_key` MERGE of different windows never collide.
- **Overlapping reads are harmless.** A lookback makes adjacent windows' *source* scans overlap, but a read-read overlap on the immutable source imposes no ordering.

Today's runtime does not yet satisfy the premise. Both the outer clamp and the DELETE currently use the *widened* write window `[run_start − k, run_end)` (§4.2), so whenever a lookback is derived, adjacent windows' write ranges overlap by k — two concurrent adjacent runs would DELETE+INSERT overlapping partitions. Window-independence therefore holds unconditionally only for zero-lookback models today, and extends to lookback models once the exact-clamp design lands (or with k-separated scheduling in the interim). The same caveat attaches permanently to any **declared source-lateness margin** (§8.6 axis (b)): its whole purpose is to *re-write* earlier partitions on later runs, so late-data reprocessing makes adjacent runs' writes overlap *by design* — a model using it trades out-of-order freedom for lateness tolerance exactly where the margin applies.

Consequently **every relaxation worked in this document — transparent subquery/CTE (Part 3), `UNION ALL` streams (Part 2), fact JOIN lookup and the bounded interval join (Part 5), and the bounded-`RANGE` window (Part 8) — is window-independent, given exact output clamps.** Each reads only source rows (its window ± a source-side margin) and writes only its own partitions. All of them may be run out of order and in parallel. This is not a coincidence: it is the same monotone/linear frontier (§7.3) — the operators that commute with the delta are exactly the ones whose per-window output does not depend on other windows.

11.3 What forces sequential execution

Only shapes that read *computed* cross-window state are sequential, and they are already named elsewhere in this audit:

- **Cumulative aggregates** (`refresh: cumulative`, D3, cumulative_aggregate.md) — the running total for window `W` *is* the accumulated state through `W−1`. Inherently ordered; this is the defining reason cumulative sits on its own refresh axis and uses `merge_into` rather than partition-scoped DELETE+INSERT.
- **Self-referential incremental models** — a model whose SQL reads its *own* prior partitions (`smelt.ref` to itself, or an engine-level read of the target table for `partition < current`). Each window consumes the last, so the chain is strict. This is the pattern the user names directly, and it is the incremental cousin of the cumulative case — the dependency is on *own output* rather than a maintained aggregate, but the ordering consequence is identical.
- **Cross-partition `UNBOUNDED PRECEDING`** windows (§8.3) — the reach that §8.3 already routes to per-partition/full recompute; when it genuinely accumulates across partitions it is the cumulative case above.

11.3a Derived, not declared

The "must run in order" property should **fall out of analysis, not a frontmatter knob** (the derive-don't-declare posture the rest of this audit takes). Both sequential triggers are statically

visible: a **self-reference** is a property of the model's ref graph (does the model's dependency set include itself?), and a **cumulative/cross-partition-unbounded** shape is exactly what the refresh axis (D3) and the Part 8 frame classifier already detect. So a model is window-independent *by default*, and becomes sequential only when the graph shows a self-edge or the refresh axis is `cumulative`. No new declared property is needed; window-independence is the derived complement of "reads its own output."

11.4 Prior art — independence is *why* engines parallelise batches

The split is externally load-bearing, not merely theoretical:

- **dbt microbatch runs batches concurrently** precisely because it treats each batch as independent (each reads its own `event_time` slice of the source); it exposes batch parallelism as a first-class knob. That is the window-independent case made operational.
- **SQLMesh** tracks intervals and can backfill independent intervals in parallel for the same reason.
- **Streaming engines** draw the opposite line for stateful operators: a running aggregate keeps ordered state (§7.3 watermarks) — the sequential case — while stateless map/filter/append stages are embarrassingly parallel (DBSP's *linear* operators, §7.3). smelt's window-independent slice is the batch analog of a stateless streaming stage; its cumulative/self-referential slice is the stateful one.

11.5 Recommendation

- **Name the property and derive it.** Add a derived model property — *window-independent* vs *ordered* — computed from (a) the ref graph (self-edge → ordered) and (b) the refresh axis / Part 8 frame reach (cumulative or cross-partition-unbounded → ordered). Default is *window-independent*.
- **Let the orchestrator use it.** A window-independent model may have its runs scheduled out of order and in parallel (disjoint-partition writes make this safe, §11.2); an ordered model must be run as a strict forward sequence. This is an execution-planning signal, adjacent to but distinct from the eligibility gates — a model can be eligible *and* ordered.
- **Surface ordering in diagnostics.** When a model is *ordered*, say *why* (self reference / cumulative), so a user who expected parallel backfill understands the constraint — the same legibility posture as the join and window rejections.

Open questions

The open questions specific to the monotonicity primitive now live in **§6.7** (column nullability at the `smelt-logical` layer, offset folding, the static-vs-declared boundary, verdict-struct shape). Several earlier entries here about "detecting independent partitionability" and "identifying the driving fact" are answered by Part 6 (one primitive, three consumers) and have moved to §6.7; the entries below are the ones that remain genuinely cross-condition.

- **Aggregating-branch unions** (Strategy B): worth it, or steer users to a CTE that unions raw events then aggregates once at the outer select (which Strategy A already handles)?
- **Static-seed branches** (case 2, constant event-time): reject, or model as a once-computed contribution to a single partition?
- **`UNION/INTERSECT/EXCEPT`:** the algebra distributes (§2.2/§2.3), but demand is unclear. Gate on a real use-case before building.
- **Detecting "independently partitionable" / identifying the driving fact** — *moved to §6.7.* Both reduce to the one monotonicity primitive (§6.4); the residual "how much statically vs. by declaration" choice is the §6.7 static-vs-declared open question.

- **Subquery body classification (B4/E2):** can "transparent" (project/filter/ rename only) be reliably distinguished from aggregating / order-sensitive bodies by static analysis of the subquery SELECT, or does the safe slice need a whitelist of recognised shapes? (§3.2)
- **CTE parity (B4/E2):** the derived-table and CTE spellings are the same query (§3.3) — should the fix unify them by classifying CTE bodies, and does closing the current CTE bypass risk newly-rejecting queries that build today?
- **Group-aligned aggregating subqueries:** an aggregation whose `GROUP BY` key ⊇ `partition_column` is window-local and safe (§3.2 row 3) — is it worth supporting directly, or steer users to the flat aggregate the outer select can already express?
- **Classifier returns a pushdown depth, not a boolean (Part 4):** how much of the "deepest safe injection point" walk can reuse the existing `source_bounds`/`temporal` analysis versus needing a new operator-by-operator pass? Is a per-source injection point always resolvable statically, or are there shapes where we must fall back to the outer clamp?
- **Retiring the outer clamp when there is no lookback (§4.3/§4.5):** is it safe to drop the outer `inject_time_filter` for the transparent slice, or should the outer clamp stay as a cheap correctness backstop even when redundant with a source filter on the same window?
- **One bound derivation instead of two (§4.5):** unifying the output-clamp window and the per-source bound (`execute.rs:895` vs `:913`) into a single per-source derivation — worth doing as part of this work, or a follow-on refactor once the classifier lands?
- **Migrating to exact output clamps (§4.2 / §8.2 / §11.2):** today's runtime widens both the clamp *and* the DELETE by the derived lookback, which re-writes margin rows from clipped scans (the confirmed §4.2 under-read) and makes adjacent runs' writes overlap (§11.2). Is the exact-clamp design adopted wholesale — and what then carries the late-data use case, whose *point* is to re-write earlier partitions (§8.6 axis (b))?
- **Join hazard as a design constraint (Part 5):** the timeseries-dimension-as- lookup misfilter (§5.2, J3, 400 violating rows) is not treated as a live incident to patch (smelt is early-stage) but as a **constraint the eligibility model must satisfy**: whatever gate lands must window only the driving fact, so J3 goes to 0 by construction. The open design choice is *how* the driving fact is identified — inferred, or declared (see next question).
- **Reuse of declared `joins:` cardinality (Part 5):** the planner already trusts declared cardinality for join elimination (§20E caveat). Should incremental eligibility reuse that same declaration to license fact-only pushdown, and does leaning on an unverified declaration for *correctness* (not just optimisation) raise the stakes of the §20E soundness caveat?
- **Cumulative vs. incremental boundary (Part 8 / Part 10):** a cross-partition `UNBOUNDED PRECEDING` running total is a *cumulative aggregate* (`refresh: cumulative`, D3, cumulative_aggregate.md), not an incremental one (§8.3). Where exactly should the window-cluster classifier hand off to the cumulative refresh — and should it *route* such a model to cumulative automatically, or reject-and-suggest? The two workstreams need one agreed line.
- **Run↔partition granularity check (Part 10):** the `g_run ≥ g_part` invariant (§10.2) is unchecked today. Should it be a hard validation, or should smelt *auto-coarsen* the run window to the partition granularity when a finer cadence is configured? And is deriving `g_part` from the partition-column transform unit (via the Part 6 primitive) sufficient, or are there partition columns whose granularity is not statically legible?
- **Self-referential incremental models (Part 11):** are models that read their own prior partitions (`smelt.ref` to self) in scope at all, or a non-goal that should be steered to `refresh: cumulative`? If in scope, the *ordered* execution property (§11.3a) must be derived and enforced (no parallel backfill); if a non-goal, the self-edge should be a named rejection rather than a silently mis-parallelised build.
- **`FOLLOWING` frames / forward reach (Part 8, §8.3):** a bounded `RANGE … INTERVAL 'a' FOLLOWING` frame has a derivable forward reach in principle, but the settledness problem is new — window `W` differs from a later full refresh until the source is complete through `hi + a`. Watermark- style delay, or tail-rewrite (with its §4.2 composition trap)? Also `source_bounds` Form A currently parses `PRECEDING` frames only.
- **Scalar subqueries over bounded sources (Part 1 addendum):** gate them with an E2-style rejection that names the construct, or teach `inject_source_filters` to leave refs inside

scalar subqueries un-windowed (they are window-invariant lookups by construction, like the §5.4 non-driving join inputs)?
- **`GROUPING SETS/ROLLUP/CUBE` (Part 1 addendum):** reject super-aggregate grouping outright for incremental models, or admit exactly the grouping sets in which *every* set contains `partition_column` (the others produce the `NULL`-partition cross-window rows)?
- **Property tests vs. single DuckDB examples (validation methodology):** each deep-dive is currently backed by a hand-written DuckDB harness with a fixed fixture (§2.3/§3.5/§5.5/§8.5). Should the incremental ≡ full invariant instead be a *property test* over generated models/data (the shape of the existing `type_property_tests` / `nullability_property_tests` oracles), so the safe slice is checked against many random inputs rather than one curated dataset — and is that worth building *before* the monotonicity primitive lands, as the oracle its red-green tests run against?

Non-goals

- Broadcast/dimension branches that must appear in every partition (case 3) — that is a JOIN, not a set operation.

Conditions worked (formerly “future stubs”)

The original rejection catalogue (Part 1) is now fully worked. Three smaller items recorded since remain unworked: the two Part 1 non-rejection addenda of 2026-07-02 (scalar subqueries over bounded sources; `GROUPING SETS/ROLLUP/CUBE`) and the `FOLLOWING`-frame mirror (§8.3) — each has an Open-questions entry. Each entry below is its own Part-2-style deep-dive — *why rejected → correctness law or mechanical limit → safe relaxation → recommendation* — and points at the Part that resolves it. The next step is not more analysis but implementation: turning the settled Parts into specs and plans (the monotonicity primitive is the shared first phase — see the Plan link in the header).

- ~~**E1 set operations (`UNION ALL`)**~~ — **worked in Part 2.** A mechanical injection-point limitation, not an algebraic one; ship the single-stream `UNION ALL` slice with a wrap-and-filter on the projected `event_time`.
- ~~**B4 subquery in `FROM`**~~ — **worked in Part 3.** Unlike E1 this is *not* an injection-depth problem (outer-SELECT injection already works); it is predicate-pushdown validity through the subquery body, plus a CTE-vs-subquery syntax inconsistency to unify.
- ~~**Joins**~~ — **worked in Part 5.** The inverse case: never gated, not universally safe. Fact JOIN lookup is safe; a second clock (timeseries dim / second fact) or a fan-out is not. Needs a driving-fact identifier and fact-only source filtering.
- ~~**B1 window functions + C1 `LAG/LEAD` + the two-layer lookback (§4.3 row 3)**~~ — **worked in Part 8.** One cluster, one phenomenon: a frame reaching outside the run window forces a widened scan bound *plus* an exact output clamp (§4.3 row 3). Only a bounded `RANGE INTERVAL` frame yields a derivable lookback k; `ROWS`/`GROUPS`/bare `LAG`/`LEAD` (C1) have no finite *time* bound, and `UNBOUNDED PRECEDING` stays per-partition. Relaxes B1’s `PARTITION BY ⊇ partition_column` gate to admit cross-partition bounded-`RANGE` windows via the Part 6 primitive + the derived margin. Empirically confirmed (W1=0 safe, W2/W3/W4 hazards, §8.5).
- ~~**B5 non-determinism — split the bucket**~~ — **worked in §9.1.** Run-deterministic (`NOW`/`CURRENT_DATE`/`CURRENT_TIMESTAMP`) vs. row-nondeterministic (`RANDOM`/`UUID`) is not one class; admit the former by pinning to a single run-shared compile-time constant (shared with the full-refresh oracle), keep rejecting the latter.
- ~~**B2 `HAVING` / B6 `DISTINCT` / B3 `LIMIT`**~~ — **worked in §9.3.** `LIMIT` never commutes (keep gated); `DISTINCT` only when its key ⊇ `partition_column` (defer to group-aligned work); `HAVING` is safe-by-default when `GROUP BY` key ⊇ `partition_column`. All three ride on the §9.4 partition-alignment check.
- ~~**A4 `partition_column` in `GROUP BY`**~~ — **worked in §9.4.** Stays a correctness law; the work is relocating its evaluation to the per-branch (§2.6) and per-body (§3.6) scopes and exposing

its verdict as the shared partition-alignment signal §9.2/§9.3 depend on. Likely lands *first* as shared infrastructure.
- ~~**Non-additive aggregates — an apparent missing rejection**~~ — **worked in §9.2; the proposed B7 gate is withdrawn (2026-07-02)**. Snowflake/BigQuery exclude `MEDIAN`/`PERCENTILE_*`/exact `COUNT(DISTINCT)` because their delta engines merge partial aggregates; smelt's A4-aligned whole-partition rebuild recomputes every group in full, so holistic aggregates are safe here (confirmed empirically, 0 violations, §9.2). The decomposability whitelist migrates to the cumulative-aggregate spec, where merging makes it load-bearing.

References

External prior art cited in Parts 6–7. Grouped by theme; every peer-reviewed entry was confirmed against at least one authoritative index (DOI, arXiv, dblp, or official venue). Two items are flagged as non-canonical where noted.

Theory — incremental maintenance, monotonicity, and limits

- **Monotone = incrementally-maintainable (CALM).** Hellerstein, "The Declarative Imperative," *SIGMOD Record* 39(1), 2010 — doi:10.1145/1860702.1860704 (the CALM conjecture). Ameloot, Neven & Van den Bussche, "Relational Transducers for Declarative Networking," PODS 2011 / *JACM* 60(2), 2013 — arXiv:1012.2858 (the proof: monotone queries are *exactly* the coordination-free class). Hellerstein & Alvaro, "Keeping CALM," *CACM* 63(9), 2020 — doi:10.1145/3369736 (accessible synthesis). Monotone/non-monotone operator boundary: Abiteboul, Hull & Vianu, *Foundations of Databases*, 1995 — webdam.inria.fr/Alice.
- **Incremental view maintenance (classic).** Blakeley, Larson & Tompa, "Efficiently Updating Materialized Views," SIGMOD 1986 — doi:10.1145/16894.16861 (irrelevant updates). Gupta, Mumick & Subrahmanian, "Maintaining Views Incrementally," SIGMOD 1993 — doi:10.1145/170035.170066 (counting algorithm). Gupta & Mumick, "Maintenance of Materialized Views: Problems, Techniques, and Applications," *IEEE Data Eng. Bull.* 18(2), 1995. Quass, Gupta, Mumick & Widom, "Making Views Self-Maintainable for Data Warehousing," PDIS 1996 (self-maintainability via key/RI constraints — the provenance argument behind tracing `event_time` to a source key).
- **DBSP / differential dataflow (modern, constructive).** Budiu, McSherry, Ryzhyk & Tannen, "DBSP: Automatic Incremental View Maintenance for Rich Query Languages," *PVLDB* 16(7), 2023 (Best Paper) — doi:10.14778/3587136.3587137 / arXiv:2203.16684 (linear vs bilinear operator taxonomy = smelt's safe slice). McSherry, Murray, Isaacs & Isard, "Differential Dataflow," CIDR 2013. Murray et al., "Naiad," SOSP 2013 — doi:10.1145/2517349.2522738.
- **Streaming / watermarks.** Akidau et al., "The Dataflow Model," *PVLDB* 8(12), 2015 — doi:10.14778/2824032.2824076 (watermark = monotone completeness bound). Akidau et al., "MillWheel," *PVLDB* 6(11), 2013. Begoli et al., "Watermarks in Stream Processing Systems," *PVLDB* 14(12), 2021 — doi:10.14778/3476311.3476389.
- **Decidability limits.** Chandra & Merlin, "Optimal Implementation of Conjunctive Queries," STOC 1977 — doi:10.1145/800105.803397 (CQ equivalence decidable, NP-complete). Trakhtenbrot's theorem (equivalence of full relational algebra/SQL undecidable) — Libkin, *Elements of Finite Model Theory*, 2004. Rice, "Classes of Recursively Enumerable Sets…," *Trans. AMS* 74(2), 1953 — doi:10.1090/S0002-9947-1953-0053041-6. Richardson, "Some Undecidable Problems Involving Elementary Functions," *JSL* 33(4), 1968 — doi:10.2307/2271358 (monotonicity of elementary expressions undecidable → whitelist is mandatory). Wang, "The Undecidability of the Existence of Zeros of Real Elementary Functions," *JACM* 21(4), 1974 — doi:10.1145/321850.321856.

Predicate pushdown & monotone-expression detection

- **Pushdown laws.** Garcia-Molina, Ullman & Widom, *Database Systems: The Complete Book* (2e), 2009, Ch. 16 §16.2 (the canonical σ-commutation law set). Selinger et al., "Access Path Selection…" (System R), SIGMOD 1979 — doi:10.1145/582095.582099. Hellerstein & Stonebraker, "Predicate Migration," SIGMOD 1993 — doi:10.1145/170036.170078. Empty-grouping-set caveat corroborated by PrestoDB PR #11297.
- **Monotone-expression detection (the closest production analogs of the Part 6 primitive).** ClickHouse `IFunctionBase::getMonotonicityForRange` + the `Monotonicity` verdict struct — `src/Functions/IFunction.h`; mechanism + whitelist in Altinity, "Learning to appreciate monotonic functions in ClickHouse". Apache Iceberg partition transforms (`preserves_order` + `project(predicate)`) — table spec, PyIceberg `transforms.py`. Delta Lake generated columns (order-preserving generation-expression whitelist). Range-rewrite framing: Bolenok, "Sargability of monotonic functions".
- **Negative baseline (won't reason about monotonicity).** Oracle VLDB & Partitioning Guide §3.1; PostgreSQL §5.12 Table Partitioning; DuckDB zonemaps/statistics propagation.

Industry incremental/materialized-view engines (published eligibility rules)

- **Databricks** — Enzyme incremental refresh; error class `MATERIALIZED_VIEW_NOT_INCREMENTALIZABLE`; AUTO CDC / `APPLY CHANGES`. Spark Structured Streaming watermarks & joins.
- **Snowflake Dynamic Tables** — supported queries, refresh modes.
- **BigQuery materialized views** — create, intro.
- **Apache Flink** — changelog processing; internal `FlinkRelMdModifiedMonotonicity` (FLINK-34702).
- **Materialize** — temporal filters, `now`/`mz_now`.
- **Feldera/DBSP** — SQL docs (DBSP paper above).
- **ClickHouse MVs** — incremental materialized view.
- **dbt** — incremental overview, microbatch. **SQLMesh** — model kinds, incremental by time. **cube.dev** — pre-aggregations.
- **Window-function eligibility & derived-vs-declared lookback (Part 8).** Spark Structured Streaming watermarking & state eviction — Databricks, "Feature Deep Dive: Watermarking in Structured Streaming", Spark Structured Streaming Programming Guide §window operations & watermarking. Flink `OVER`-window append-only vs Top-N updating & the rowtime-`ORDER BY` requirement — Confluent, "Changelog processing in Flink SQL". Snowflake Dynamic Tables window functions as blocking operators — Optimize queries for incremental refresh, supported queries. Databricks Enzyme `WINDOW_WITHOUT_PARTITION_BY` — `MATERIALIZED_VIEW_NOT_INCREMENTALIZABLE` error class. Declared source-lateness knobs (axis (b) of §8.6): dbt microbatch `lookback` / microbatch strategy; SQLMesh incremental-by-time interval tracking. ClickHouse window view (watermark-driven tumbling/hopping).

Non-canonical (illustrative only): Fink, "On Monotonicity in Relational Databases…," Palantir Engineering, 2018 (blog); US Patent 8,176,035 (monotonicity tracking — existence evidence, not a proof). The scalar order-preservation lemma ($a \le x < b \Rightarrow f(a) \le f(x) < f(b)$ for monotone f) has no single canonical paper; it is standard order theory realised by the zone-map/micro-partition pruning systems above.